TECHNICAL DOCUMENTATION

Caliper

A browser 3D model viewer that measures — explained end to end, from the first line of HTML to the deployed site.

Version 1.0.1

github.com/satyaidk/Caliper

React 18 · TypeScript 5 · three.js r169 · Vite 5 · Zustand 4

Written for a frontend developer learning by taking a real app apart

BEFORE YOU START

How to read this

This document assumes you can write HTML, CSS and some JavaScript, and that React and 3D graphics are new or half-known. Everything else is explained.

It is organised so you can read it in one pass or dip into it while the code is open beside you. The five parts do different jobs:

Part I	What the app is and what every dependency is for. Read this first, in order.
Part II	Every file in the repository, explained. Use it as a reference — jump to the file you are looking at.
Part III	The ideas underneath: React patterns, state management, 3D graphics, the measuring maths. Read in order.
Part IV	Build tooling, git, CI, deployment. Read when you want to ship something of your own.
Part V	Exercises, a glossary, and where to read more.

> **THE MOST USEFUL WAY TO USE THIS**
>
> Do not read it like a novel. Open the file being discussed in your editor, read the section, then change something and watch what breaks. A concept you have broken and fixed is a concept you own. Part V has ten exercises designed exactly for that.

Conventions

Code you can type is in `monospace`. File paths are written from the repository root: `src/store/useViewer.ts`. Keys are shown as `M`. Where a snippet is shortened, `…` marks what was cut. Every code sample is real code from the repository, not pseudocode — you can search for it.

CONTENTS

What is in here

PART I

Orientation

What the application is, which libraries it leans on and why, and how to get it running on your machine in under a minute.

CHAPTER 1

What Caliper is, and what it does

Someone emails you a `.step` file. You have no CAD software. Caliper opens it in a browser tab and tells you how big it is.

1.1 The problem it solves

3D model files come in dozens of incompatible formats, and most of them need expensive desktop software to open. If all you need is to check a dimension — will this bracket fit, what is the diameter of that hole — that is a heavy price. Caliper is the light version of that job: open the file, read the numbers, close the tab.

Everything happens on the visitor's own machine. The file is never uploaded. That is not only a privacy claim — it is an architectural fact that shapes the whole codebase, because there is no server, no API, no database and no authentication anywhere in this project. It is a pure frontend application, which is exactly what makes it a good thing to learn from.

1.2 What a user can do

CAPABILITY	HOW IT WORKS
Open a model	Drag and drop, a file picker, or a `?model=` URL
Read its size	Width, height and depth in mm, cm, m or inches
Measure a distance	Click two points; the span is also split into X, Y and Z
Measure a diameter	Click three points around an arc; a circle is fitted to them
Look at it	Orbit, pan, zoom; shaded, wireframe or shaded with edges
Select a part	Click it for its name and triangle count
Save a picture	PNG at twice screen resolution

1.3 The three format pipelines

Twenty-two file extensions are supported, and they fall into three groups that reach the screen by genuinely different routes. Understanding this grouping explains most of the `src/viewer/loaders/` folder.

PIPELINE	EXTENSIONS	WHAT HAS TO HAPPEN
Mesh	`stl obj ply off gltf glb fbx dae 3ds 3mf amf wrl`	The file already contains triangles. Parse them into memory and draw.
CAD B-rep	`step stp iges igs brep brp fcstd 3dm`	The file contains mathematical *surfaces*, not triangles. A geometry kernel compiled to WebAssembly must *tessellate* them — approximate each curved surface with a mesh — before anything can be drawn.
BIM	`ifc bim`	Building models. Geometry arrives already grouped into walls, doors and slabs, each with its own colour and identity.

> **WHY THIS MATTERS FOR THE DOWNLOAD SIZE**
>
> The CAD and BIM kernels are 5–12 MB WebAssembly builds. They are fetched from a CDN the first time someone opens a file that needs them, never bundled. A visitor who only ever opens an `.stl` downloads none of it. This is **lazy loading**, and it is one of the most valuable habits in frontend work: pay for a feature only when it is used.

1.4 What was deliberately left out

Version 1.0 is small on purpose. Angle measurement, section planes, a scene tree, exploded views, animation playback and file export were all built and then removed before release, because a tool that does two things clearly beats one that does nine things confusingly. The roadmap in the README lists them in the order they are worth adding back.

This is worth internalising as a habit: **the hard part of design is subtraction.** Adding a feature is easy; deciding it does not belong in version one is the skill.

CHAPTER 2

The stack, and why each piece is there

Five runtime dependencies, five development ones. Every entry in `package.json` earns its place; here is what each one does.

2.1 Runtime dependencies

PACKAGE	SIZE	WHAT IT DOES AND WHY IT IS HERE
`react` `react-dom`	~45 kB gz	Turns application state into DOM. Every panel, button and readout is a React component. `react-dom` is the part that actually touches the browser; React itself is renderer-agnostic.
`three`	~144 kB gz	The 3D engine. Wraps WebGL — the browser's low-level GPU API — in cameras, meshes, materials and lights. Writing this app in raw WebGL would take thousands of lines of shader plumbing.
`zustand`	~1 kB gz	State management. One shared object that any component can read from and write to, without passing props down through five layers. Chapter 14 explains it from scratch.
`fflate`	~10 kB gz	A zip decompressor. Used by exactly one loader: a FreeCAD `.FCStd` file is a zip archive with the geometry inside.

2.2 Development dependencies

PACKAGE	WHAT IT DOES
`vite`	The dev server and the production bundler. Chapter 21.
`@vitejs/plugin-react`	Teaches Vite to compile JSX and enables hot reloading that preserves component state.
`typescript`	Type checking. Produces no runtime code — it is a safety net at build time only.
`@types/react`, `@types/react-dom`, `@types/three`	Type definitions for libraries written before TypeScript, or shipped without types.

> **NOTICE WHAT IS ABSENT**
>
> No React Router. No Redux. No Tailwind. No component library. No test framework. No ESLint. Each of those is a reasonable choice on some project — none of them earns its place on this one, and every dependency you do not add is one you never have to upgrade, patch or debug. When you start your own project, begin from zero and add only what a real problem forces you to add.

2.3 The one architectural rule

There is a single hard rule in this codebase, and almost everything else follows from it:

THE RULE

`src/viewer/` never imports React.

`src/components/` never imports three.js.

The 3D engine is a plain TypeScript class that knows about cameras and meshes and nothing about user interfaces. The React components know about buttons and panels and nothing about WebGL. They meet in exactly two places: the Zustand store, and a small set of callbacks the engine fires when something happens.

This separation is why the engine can be understood without knowing React, and the UI can be understood without knowing 3D graphics. When you build your own applications, look for the equivalent line — the boundary between "the thing that does the work" and "the thing that shows it" — and defend it.

The two halves of the application and the single place they touch. An arrow that skipped the store — a component importing the Engine directly, say — would be the first crack in the design.

CHAPTER 3

Getting it running

3.1 What you need

Node.js 22 or newer. The repository pins 24 in `.nvmrc`, which is the version continuous integration uses. Node ships with `npm`, which is all the tooling you need.

3.2 Four commands

```
# install every dependency listed in package.json
npm install

# start the dev server with hot reloading — http://localhost:5173
npm run dev

# type-check the whole project without producing any files
npm run typecheck

# run the unit tests — Node's own test runner, no framework to install
npm test

# produce the deployable bundle in dist/
npm run build
```

These are defined in `package.json` under `scripts`. `npm run dev` is what you will use ninety-five per cent of the time: save a file, and the browser updates in well under a second without losing the model you have open.

3.3 What to try first

1. Run `npm run dev` and open the address it prints.
2. Click **Load a sample STL**. A robot head appears on a build plate.
3. Drag to orbit, scroll to zoom, right-drag to pan.
4. Press `M`, then click two points on the model. A dimension line appears with a number on it.
5. Press `W` for wireframe, `Q` to go back.
6. Open `src/components/Panel.tsx`, change the word `"Width"` to `"Across"`, and save. Watch it change in the browser without a reload.

That last step is the loop you will spend your career in. Everything else in this document is detail on top of it.

PART II

The codebase, file by file

Every file in the repository, what it is responsible for, and the parts of it worth reading closely. Use this as a reference: find the file you have open and read its section.

CHAPTER 4

The repository map

```
Caliper/
├── .github/workflows/
│   └── ci.yml            typecheck + build on every push (ch. 24)
├── docs/          this document and its source
├── public/        copied to the site root untouched
│   ├── favicon.svg  icon.svg      browser tab and app icons
│   ├── manifest.webmanifest       makes it installable (ch. 5)
│   ├── og.png                     social preview image
│   └── robots.txt  sitemap.xml    search engine hints
├── src/
│   ├── components/   React. No three.js in here.
│   │   ├── EmptyState.tsx  Fallback.tsx  Gizmo.tsx
│   │   ├── Icons.tsx       Panel.tsx     StatusStrip.tsx
│   │   ├── Toasts.tsx      ToolRail.tsx  TopRail.tsx
│   │   └── Viewport.tsx
│   ├── hooks/        reusable stateful behaviour
│   │   ├── useFileDrop.ts  useHotkeys.ts  useMediaQuery.ts
│   ├── lib/          pure functions, no React, no three.js
│   │   ├── cdn.ts  format.ts  formats.ts
│   ├── store/
│   │   └── useViewer.ts     the single source of truth
│   ├── styles/
│   │   ├── tokens.css       colour and type variables
│   │   ├── base.css         reset and page layout
│   │   └── ui.css           every component's styling
│   ├── viewer/       three.js. No React in here.
│   │   ├── Engine.ts        the orchestrator
│   │   ├── Stage.ts         plate, grid, lights
│   │   ├── Annotations.ts   measurement lines
│   │   ├── Bracket.ts       the selection marker
│   │   ├── measure.ts       snapping and circle fitting
│   │   ├── analyze.ts       statistics and bounds
│   │   ├── renderer-registry.ts
│   │   └── loaders/  one module per format family
│   ├── App.tsx              the layout
│   ├── main.tsx             the entry point
│   ├── types.ts             shared type definitions
│   └── vite-env.d.ts
├── index.html               the real entry point (ch. 5)
├── package.json             dependencies and scripts
├── tsconfig.json            TypeScript settings (ch. 22)
├── vite.config.ts           build settings (ch. 21)
├── vercel.json              deployment settings (ch. 25)
├── .nvmrc  .gitattributes  .gitignore
├── CHANGELOG.md  README.md
└── package-lock.json        exact dependency versions — commit this
```

4.1 Why the folders are named this way

Folder names in a React project are conventions, not rules, but these ones carry meaning worth copying:

`components/`	Things that render. If it returns JSX, it lives here.
`hooks/`	Reusable behaviour that needs React's lifecycle. Every file exports one function starting with `use`.
`lib/`	Pure functions. Give them the same input and they always return the same output. No React, no three.js, no side effects. These are the easiest things in any codebase to test and reason about.
`store/`	Shared application state.
`viewer/`	The domain — in this app, 3D. On a different app this might be `editor/` or `player/`. It is the part that would still exist if you rewrote the UI in a different framework.
`styles/`	CSS, split by scope rather than by component.

> **A TEST FOR WHETHER A FILE IS IN THE RIGHT FOLDER**
>
> Ask: "if I rewrote the interface in Vue tomorrow, would this file survive unchanged?" Everything in `lib/` and `viewer/` should survive. Everything in `components/` and `hooks/` would be rewritten. If a file cannot be sorted by that question, it is probably doing two jobs.

CHAPTER 5

The boot sequence

Three files run, in order, before anything is on screen: `index.html`, then `main.tsx`, then `App.tsx`. Follow them once and the whole application opens up.

5.1 `index.html` — the real entry point

Many React tutorials skip past this file. It is worth reading properly, because in a Vite project it is not a template — it is the actual document the browser receives, and Vite treats it as the root of the dependency graph.

The theme script

```
<script>
  (function () {
    try {
      var saved = localStorage.getItem('caliper.theme');
      document.documentElement.dataset.theme =
        saved === 'dark' || saved === 'light'
          ? saved
          : matchMedia('(prefers-color-scheme: light)').matches ? 'light' : 'dark';
    } catch (e) { /* keep the markup default */ }
  })();
</script>
```

This is a **blocking inline script in the head** — normally something to avoid, and here exactly right. It sets `data-theme` on the `<html>` element *before the browser paints anything*.

Without it, the page would paint with the default dark theme, React would boot a moment later, discover the visitor prefers light, and repaint. That white flash is called **FOUC** — flash of unstyled content — and it is the single most common polish bug in themed applications. The fix is always the same: decide the theme before first paint, in a script small enough that blocking on it costs nothing.

The `try/catch` matters too. `localStorage` throws in some private browsing modes. A crash here would leave the page blank forever, so the catch falls back to the value already in the markup.

Meta tags

`viewport`	`width=device-width, initial-scale=1` makes the page respond to phone widths. Note there is deliberately **no** `maximum-scale=1` — that would block pinch-zoom, which many people rely on to read.
`description`	What search engines show under the title.
`og:*`, `twitter:*`	Open Graph tags. When the link is pasted into Slack, WhatsApp or a social network, these decide the title, description and preview image.
`theme-color`	Colours the browser chrome on mobile. Two are declared, one per colour scheme.
`manifest`	Points at `manifest.webmanifest`, which lets the browser offer "install this app".

The mount point

```
<div id="root"></div>
<noscript>Caliper renders models with WebGL and needs JavaScript enabled.</noscript>
<script type="module" src="/src/main.tsx"></script>
```

One empty `div`. React will fill it. The `type="module"` attribute is what makes the browser treat the file as an ES module, which is how Vite serves source directly in development without bundling.

5.2 `src/main.tsx` — mounting React

```
const root = createRoot(document.getElementById('root')!);

root.render(
  <StrictMode>
    <ErrorBoundary>{hasWebGL() ? <App /> : <NoWebGL />}</ErrorBoundary>
  </StrictMode>,
);
```

Four things happen in six lines.

`createRoot`	The React 18 entry API. It hands React ownership of that `div`.
`StrictMode`	A development-only wrapper. It deliberately runs effects twice to surface bugs caused by code that assumes an effect runs once. It disappears in production builds. If something works in production but breaks in development, StrictMode is usually the reason — and it is usually right.
`ErrorBoundary`	Catches any error thrown while rendering, anywhere below it, and shows a readable message instead of a white page.
`hasWebGL()`	A capability probe. If the browser cannot give us a GPU context there is no point mounting the app at all, so a plain explanation is shown instead.

Note the `!` after `getElementById('root')`. That is TypeScript's non-null assertion: "I know this can return null, but here it cannot." Use it sparingly — it switches the type checker off for that expression — but the root element is a fair case, since if it is missing the application is unrunnable anyway.

5.3 `src/App.tsx` — the layout

`App` is the top component. It does four jobs and nothing else: it draws the layout, wires up the file input, registers the keyboard, and handles the two ways a model can arrive from outside a click.

The layout

```
<div className="shell" data-panel={panelOpen && !compact ? 'open' : 'closed'}>
  <TopRail onOpen={openPicker} onScreenshot={screenshot} />
  {!compact && <ToolRail />}

  <main className="area-stage">
    <Viewport dragging={dragging} onOpen={openPicker} />
  </main>

  {!compact && panelOpen && <div className="area-panel"><Panel /></div>}
  {!compact && <StatusStrip />}
  …
</div>
```

Two React idioms appear here that are worth naming.

Conditional rendering with `&&`. `{!compact && <ToolRail />}` renders the rail only on wide screens. It works because JavaScript's `&&` returns the right-hand side when the left is truthy, and React ignores `false`. Watch out for one trap: `{items.length && <List />}` renders a literal `0` when the array is empty, because `0` is falsy but not `false`. Always compare explicitly — `{items.length > 0 && …}`.

Data attributes for state. `data-panel="open"` lets CSS respond to application state without JavaScript computing any styles. The stylesheet does the rest: `.shell[data-panel='closed'] { grid-template-columns: … 0; }`. This keeps layout logic in CSS where it belongs.

Loading a model from the URL

```
function modelFromUrl() {
  const params = new URLSearchParams(window.location.search);
  const raw = params.get('model');
  if (!raw) return null;
  try {
    const url = new URL(raw, window.location.href);
    if (url.protocol !== 'https:' && url.protocol !== 'http:') return null;
    …
  } catch { return null; }
}
```

This is the closest thing the app has to routing, and it is also a small security lesson. Anything that comes from a URL is attacker-controlled. The protocol check rejects `javascript:`, `data:` and `file:` — schemes a fetch has no business following. Parsing with the `URL` constructor inside a `try` also rejects malformed input for free, rather than hand-rolling a parser that will have bugs.

File handlers

```
useEffect(() => {
  if (!engine || !('launchQueue' in window)) return;
  (window.launchQueue as LaunchQueue).setConsumer(async (params) => {
    const files = await Promise.all(params.files.map((h) => h.getFile()));
    if (files.length) void open(files);
  });
}, [engine, open]);
```

When Caliper is installed as an app, the operating system can hand it files that were double-clicked in a file manager. The `'launchQueue' in window` guard is **feature detection**: only Chromium browsers implement this, so the code checks rather than assuming. Never detect browsers by name; detect the feature you need.

What `void` means here

`void open(files)` calls an async function and explicitly discards the promise. It communicates "I know this is asynchronous and I am deliberately not awaiting it" — to other readers, and to lint rules that flag floating promises.

CHAPTER 6

The store — one source of truth

`src/store/useViewer.ts` is the most important file in the application. Everything the interface shows lives here, and every change to the 3D scene goes through here.

6.1 The problem it solves

The status strip at the bottom shows the model's width. So does the panel on the right. The tool rail knows which render mode is active; so does the keyboard handler. None of these components are near each other in the tree.

Without a store you would lift that state up to `App` and thread it down through props — the pattern known as **prop drilling**. It works, and it becomes miserable at about three levels deep, because every intermediate component has to accept and forward props it does not use.

A store is a box that lives outside the component tree. Components subscribe to the slices they care about.

6.2 The shape of the state

```
interface ViewerState {
  engine: Engine | null;      // the 3D engine instance
  status: 'idle' | 'loading' | 'ready';
  model: ModelInfo | null;   // name, size, triangle counts
  selection: SelectionInfo | null;
  measurements: Measurement[];
  display: Display;           // render mode, grid, shadows
  unit: UnitSystem;           // mm | cm | m | in
  theme: Theme;
  toasts: Toast[];
  …
  open(files: File[]): Promise<void>;   // actions live here too
  setMeasureTool(tool: MeasureTool): void;
  …
}
```

Note that **data and actions live in the same object**. That is a Zustand convention and a good one: a component asks for `setMeasureTool` the same way it asks for `model`, and there is one import instead of two.

6.3 Reading from the store

```
const model = useViewer((s) => s.model);
const unit  = useViewer((s) => s.unit);
```

The function passed in is a **selector**. It picks one slice, and the component re-renders only when *that slice* changes. If a measurement is added, a component that only selected `unit` does not re-render at all.

THE MISTAKE TO AVOID

`const state = useViewer();` subscribes to *everything*. In this app the frame-rate counter updates twice a second, so a component written that way would re-render twice a second forever, whether or not it shows the frame rate. Always select the narrowest slice you need.

6.4 Writing to the store

```
setUnit(unit) {
  set({ unit });     // merges into state, triggers subscribers
  persist();         // writes preferences to localStorage
},
```

`set` performs a shallow merge — you pass only the keys that changed. For state derived from the previous value, pass a function:

```
togglePanel(force) {
  set((s) => ({ panelOpen: force ?? !s.panelOpen }));
  persist();
},
```

The `??` is the **nullish coalescing** operator: use `force` unless it is `null` or `undefined`. Unlike `||` it does not treat `false` as missing, which matters here because `false` is a meaningful value.

6.5 The most important function: `open()`

Opening a file is the app's central operation. Read it once and the whole data flow becomes clear.

```
async open(files) {
  const engine = get().engine;
  if (!engine) return;

  // 1. Which of the dropped files is the model?
  const primary = pickPrimary(files);
  if (!primary) { /* toast: nothing to open */ return; }

  // 2. Show the loading state immediately.
  set({ status: 'loading', progress: null, progressNote: 'Reading file' });

  // 3. Blob URLs for every dropped file, so loaders can resolve siblings.
  const ctx = buildContext(files, (fraction, note) => set(…));

  try {
    // 4. Parse. This is where the format-specific work happens.
    const { object, hasAuthoredMaterials } = await loadModel(primary, ctx);

    // 5. Measure it — in the file's own units, before any scaling.
    const stats      = computeStats(object);
    const dimensions = computeDimensions(object);

    // 6. Hand the geometry to the engine.
    engine.setModel(object, hasAuthoredMaterials);

    // 7. Publish everything the UI needs, in one update.
    set({ status: 'ready', model: { name, ext, bytes, stats, dimensions, … } });
  } catch (error) {
    set({ status: get().model ? 'ready' : 'idle' });
    // toast explaining what went wrong, in the error's own words
  } finally {
    setTimeout(ctx.revoke, 12_000);   // release blob URLs later
  }
}
```

Several things here are worth copying into your own work.

Step 5 happens before step 6, and that ordering is load-bearing. The engine scales every model to a fixed size so lighting and the grid behave consistently. If dimensions were measured after that, the app would report the viewer's internal units instead of the file's real ones — the numbers would be meaningless. Measure first, then transform.

The `catch` restores a sensible state. If a model was already open, a failed load leaves it open. Only a failure with nothing to fall back on returns to `idle`. Error handling is not just "show a message" — it is "decide what the application is now".

The `finally` cleans up regardless. Blob URLs hold memory until revoked. The twelve-second delay exists because a glTF may still be fetching textures through those URLs after the main parse resolves.

6.6 Persisting preferences

```
function readPrefs(): Partial<Prefs> {
  try {
    const raw = localStorage.getItem(PREFS_KEY);
    return raw ? JSON.parse(raw) : {};
  } catch {
    return {};
  }
}
```

Only settings that describe *how you like to look at models* persist: render mode, grid, shadows, unit, panel visibility. Nothing tied to the file in front of you is remembered, because restoring a measurement onto a different model would be nonsense.

Every storage call is wrapped in `try/catch`. Storage throws in private modes and when a quota is exceeded, and a remembered toggle is never worth a crash.

CHAPTER 7

The React components

Ten files in `src/components/`. None of them import three.js; all of them read from the store.

7.1 `Viewport.tsx` — where React meets WebGL

This is the most interesting component in the app, because it is the seam between two worlds. React owns the DOM declaratively; three.js owns a canvas imperatively. This file negotiates that.

```
useEffect(() => {
  const canvas = canvasRef.current;
  const host   = hostRef.current;
  if (!canvas || !host) return;

  const store  = useViewer.getState();
  const engine = new Engine(canvas, {
    onFrame:   store.setFrame,
    onHover:   store.setHover,
    onSelect:  store.setSelection,
    onMeasure: store.setMeasurements,
    …
  });
  engine.resize();
  attach(engine);

  const observer = new ResizeObserver(() => engine.resize());
  observer.observe(host);

  return () => {          // cleanup — runs on unmount
    observer.disconnect();
    detach();
    engine.dispose();
  };
}, [attach, detach]);
```

Refs, not state, for DOM nodes. `useRef` gives you a mutable box whose `.current` can be changed without re-rendering. React fills it with the DOM node when the element mounts. Using state here would cause an infinite loop.

The cleanup function is not optional. The engine holds a WebGL context, GPU buffers, textures and an animation loop. Without `engine.dispose()` those leak — and browsers allow only a handful of live WebGL contexts before they start killing the oldest. Any effect that creates something must return a function that destroys it.

`getState()` versus `useViewer(…)`. Inside the effect the store is read with `useViewer.getState()`, which reads once without subscribing. Subscribing here would re-run the effect — destroying and rebuilding the entire 3D engine — every time unrelated state changed.

`ResizeObserver` beats a window listener. The canvas can change size when the panel is toggled, with no window resize at all. Observing the element catches every case.

The measurement labels — bypassing React on purpose

```
engine.setLabelSink((anchors) => {
  for (const anchor of anchors) {
    const node = nodes.current.get(anchor.id);
    if (!node) continue;
    node.style.transform =
      `translate(-50%, -50%) translate(${anchor.x}px, ${anchor.y}px)`;
    node.style.visibility = anchor.behind ? 'hidden' : 'visible';
  }
});
```

Each measurement has a floating tag showing its value. As the camera orbits, every tag must reposition — up to sixty times a second.

Routing that through React state would re-render the panel, the status strip and everything else sixty times a second. So React renders the tag elements *once* (their content only changes when a measurement changes), and the engine writes `style.transform` directly on the DOM nodes.

This is a legitimate escape hatch, and the reasoning generalises: **React for structure and content, direct DOM manipulation for high-frequency visual updates.** The rule for using it safely is that React must not also be trying to control the same property.

7.2 `Panel.tsx` — the details panel

One scrolling column, in the order the questions get asked: how big is it, what do I need to measure, what did I click, what is this file.

The file starts with two tiny local components:

```
function Row({ label, value }: { label: string; value: string }) {
  return (
    <div className="field">
      <span className="field-label">{label}</span>
      <span className="field-value">{value}</span>
    </div>
  );
}
```

Six lines, used eight times. This is composition at its most ordinary and most valuable: a name for a repeated shape. Not every component needs its own file — a component used only inside one module belongs in that module.

Early return for the empty state

```
if (!model) {
  return <aside className="panel"><p className="hint">Open a model and its size lands here.</p>
</aside>;
}
```

Returning early keeps the rest of the function free of `model?.` checks. After that line TypeScript knows `model` is not null, and every reference below is clean. This is **narrowing**, and leaning on it is one of the biggest quality-of-life wins TypeScript offers.

Naming the axes

```
<AxisRow axis="x" name="Width"  value={d.x} … />
<AxisRow axis="y" name="Height" value={d.y} … />
<AxisRow axis="z" name="Depth"  value={d.z} … />
```

A small copy decision with a large usability effect. "X 80.00 mm" makes the reader translate; "X Width 80.00 mm" answers the question directly. Interface text is part of the design, not a label you fill in afterwards.

7.3 `ToolRail.tsx` — buttons and shared behaviour

```
const MODES: { mode: RenderMode; label: string; keys: string; icon: ReactNode }[] = [
  { mode: 'shaded',    label: 'Shade the surfaces',  keys: 'Q', icon: <IconShaded /> },
  { mode: 'wireframe', label: 'Show the wireframe',  keys: 'W', icon: <IconWire /> },
  { mode: 'edges',     label: 'Shade with edges',    keys: 'E', icon: <IconEdges /> },
];
```

Data-driven rendering. The three buttons are described as data and rendered with `.map()`. Adding a fourth mode means adding a row, not writing more JSX. Whenever you find yourself copying a block of markup and changing two words, reach for this.

The exported `ToolButtons` is used twice — inside the desktop rail and inside the mobile dock — with different CSS around it. Same behaviour, two presentations, one implementation.

7.4 `Gizmo.tsx` — animating outside React

The orientation widget projects the world axes through the live camera rotation so it shows which way you are facing. That means updating every frame, which is the same problem the measurement tags had, solved the same way.

```
useEffect(() => {
  if (!engine) return;
  let raf = 0;

  const tick = () => {
    raf = requestAnimationFrame(tick);
    engine.viewQuaternion(quaternion);
    inverse.copy(quaternion).invert();

    ARMS.forEach((arm, index) => {
      projected.copy(arm.vector).applyQuaternion(inverse);
      const x = CENTRE + projected.x * RADIUS;
      const y = CENTRE - projected.y * RADIUS;
      node.setAttribute('transform', `translate(${x} ${y})`);
      …
    });
  };

  raf = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(raf);   // stop the loop on unmount
}, [engine]);
```

`requestAnimationFrame` asks the browser to call you before the next repaint — roughly sixty times a second, and automatically paused when the tab is hidden. Never use `setInterval` for animation: it does not sync with the display and keeps running in background tabs.

The cleanup cancelling the frame is essential. Without it the loop keeps running after the component unmounts, holding references to dead DOM.

Sorting by depth

```
depths.sort((a, b) => a.z - b.z);
for (const { index } of depths) stack.appendChild(armRefs.current[index]);
```

SVG has no z-index; it paints in document order. To make the nearest axis draw on top, the elements are physically reordered each frame. `appendChild` on an existing node moves it rather than copying it.

7.5 The remaining components

FILE	WHAT IT DOES	WORTH NOTICING
`TopRail.tsx`	Logo, filename, and four actions.	Takes `onOpen` and `onScreenshot` as props rather than reaching into the store, because those need DOM access that lives in `App`.
`StatusStrip.tsx`	Width, height, depth, triangles, render mode.	Early-returns a different strip when no model is open, instead of littering the markup with conditionals.
`EmptyState.tsx`	The first screen: headline, sample buttons, format table.	The format table is *derived* from the format list, not typed out. A hand-written copy would drift the first time a format was added.
`Toasts.tsx`	Transient messages.	`role="status"` and `aria-live="polite"` make screen readers announce them without interrupting.
`Icons.tsx`	Every icon as an inline SVG component.	A shared `base()` function fixes the viewBox and stroke so all icons look like one set. `stroke="currentColor"` makes them inherit text colour, so hover and active states come free.
`Fallback.tsx`	Error boundary and the no-WebGL screen.	The only class component in the codebase — error boundaries have no hook equivalent.

Why `ErrorBoundary` must be a class

```tsx
export class ErrorBoundary extends Component<Props, State> {
  state: State = { error: null };

  static getDerivedStateFromError(error: Error): State {
    return { error };
  }

  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error('Caliper stopped:', error, info.componentStack);
  }
  …
}
```

React has never shipped a hook that catches render errors. This is the one case where a class component is not legacy code but the only option. It is worth having exactly one in every application — without it, a single thrown error blanks the entire page.

CHAPTER 8

The hooks

A custom hook is a function that uses other hooks. That is the whole definition. It exists to give a piece of stateful behaviour a name and let two components share it.

8.1 `useMediaQuery.ts`

```ts
export function useMediaQuery(query: string): boolean {
  const [matches, setMatches] = useState(
    () => typeof window !== 'undefined' && window.matchMedia(query).matches,
  );

  useEffect(() => {
    const mql = window.matchMedia(query);
    const onChange = () => setMatches(mql.matches);
    onChange();
    mql.addEventListener('change', onChange);
    return () => mql.removeEventListener('change', onChange);
  }, [query]);

  return matches;
}

export const useIsCompact = () => useMediaQuery('(max-width: 860px)');
```

Two details repay attention.

The lazy initialiser. `useState(() => …)` passing a *function* means the expression runs only on the first render. `useState(window.matchMedia(query).matches)` would call `matchMedia` on every single render and throw the result away.

Matching media in JavaScript, not just CSS. CSS can hide the tool rail on a phone, but React needs to know so it can *not render it at all* — a hidden component still mounts, still runs its effects, and still costs memory.

8.2 `useFileDrop.ts`

Window-wide drag and drop, with recursive folder reading. The trickiest part is not the drop — it is knowing when the drag has actually left.

```ts
const depth = useRef(0);

const onDragEnter = (e: DragEvent) => { depth.current++; setDragging(true); };
const onDragLeave = (e: DragEvent) => {
  depth.current = Math.max(0, depth.current - 1);
  if (depth.current === 0) setDragging(false);
};
```

Drag events bubble. Moving the pointer from the page onto a button inside it fires `dragleave` for the page and `dragenter` for the button — so a naive implementation flickers the drop overlay constantly. Counting enters and leaves and only hiding at zero fixes it. This is a classic browser-quirk fix, and the kind of thing worth recognising rather than rediscovering.

Reading dropped folders

```
const readEntry = async (entry, path, out) => {
  if (entry.isFile) {
    const file = await new Promise((resolve, reject) => entry.file(resolve, reject));
    Object.defineProperty(file, 'webkitRelativePath', { value: path + file.name });
    out.push(file);
    return;
  }
  const reader  = entry.createReader();
  const entries = await new Promise((resolve) => reader.readEntries(resolve, () => resolve([])));
  for (const child of entries) await readEntry(child, `${path}${entry.name}/`, out);
};
```

A glTF file references its textures by relative path, so dropping the folder has to preserve that structure. The function **recurses** — it calls itself for each subdirectory — and builds up the path as it descends.

The `new Promise(…)` wrappers convert old callback-style browser APIs into promises so they can be used with `await`. That conversion is called **promisifying** and you will do it often.

8.3 `useHotkeys.ts`

```
const EDITABLE = new Set(['INPUT', 'TEXTAREA', 'SELECT']);

const onKey = (event: KeyboardEvent) => {
  const target = event.target as HTMLElement | null;
  if (target && (EDITABLE.has(target.tagName) || target.isContentEditable)) return;

  // Space and Enter already belong to whatever control has focus.
  if ((key === ' ' || key === 'enter') && target?.tagName === 'BUTTON') return;
  …
};
```

Global keyboard shortcuts have two failure modes, and both are guarded here. Typing "m" in a text field must not switch to the measure tool. And pressing space while a button has focus must not both activate the button and fire a shortcut.

The store is read with `useViewer.getState()` rather than a subscription, so the listener is registered once instead of being torn down and rebuilt whenever state changes.

CHAPTER 9

The helpers in `lib/`

Pure functions. No React, no three.js, no side effects. The easiest code in the project to read and the easiest to reuse.

9.1 `format.ts` — turning numbers into text

```ts
export function bytes(n: number): string {
  if (n < 1024) return `${n} B`;
  const units = ['KB', 'MB', 'GB'];
  let v = n / 1024, i = 0;
  while (v >= 1024 && i < units.length - 1) { v /= 1024; i++; }
  return `${v < 10 ? v.toFixed(1) : Math.round(v)} ${units[i]}`;
}
```

Note the precision rule: one decimal below ten, none above. "9.4 MB" is useful; "9.43871 MB" is noise, and "412.7 MB" is false precision. Formatting is a design decision.

The floating-point trap in `dim()`

```ts
export function dim(value: number, unit: UnitSystem): string {
  const v = value * UNIT_FACTOR[unit];
  if (v === 0) return `0 ${unit}`;
  // Nudge off the boundary first: 25.4 mm converts to 0.99999… inches,
  // and without this it would print with four decimals instead of two.
  const abs = Math.abs(v) * (1 + Number.EPSILON * 8);
  const digits = abs >= 1000 ? 0 : abs >= 100 ? 1 : abs >= 1 ? 2 : 4;
  return `${v.toFixed(digits)} ${unit}`;
}
```

Computers cannot represent most decimals exactly. 25.4 divided by 25.4 should be 1, but in binary floating point it lands a hair below — so the "is it at least 1?" test fails and the number prints as `1.0000` instead of `1.00`. The nudge by a few `EPSILON` pushes it back over the line. Every application that formats numbers eventually meets this bug.

9.2 `formats.ts` — the format registry

```ts
export const FORMATS: FormatSpec[] = [
  { ext: 'stl',  label: 'Stereolithography', pipeline: 'mesh' },
  { ext: 'obj',  label: 'Wavefront', pipeline: 'mesh', companions: ['mtl', 'png', 'jpg'] },
  …
];

const BY_EXT = new Map(FORMATS.map((f) => [f.ext, f]));
```

One array is the source of truth. Everything else is derived from it: the lookup map, the `accept` attribute on the file input, the grouped table on the empty state, and the count shown in its header.

> **DERIVE, NEVER DUPLICATE**
>
> An earlier version of this file had a hand-written showcase list beside the registry. The empty state advertised "18 formats" while the registry held 22 — the two had drifted apart silently. The fix was to delete the second list and compute it. Any time you write the same fact twice, you have created a bug that is waiting for someone to edit only one copy.

The `Map` matters too. Looking an extension up in an array with `.find()` scans it every time; a `Map` is a constant-time hash lookup. At 22 entries this is irrelevant — but the habit is right, and the intent reads more clearly.

9.3 `cdn.ts` — loading code on demand

```ts
export const CDN = {
  occt:     'https://cdn.jsdelivr.net/npm/occt-import-js@0.0.23/dist/',
  webIfc:   'https://cdn.jsdelivr.net/npm/web-ifc@0.0.57/',
  …
} as const;

const pending = new Map<string, Promise<void>>();

export function loadScript(url: string): Promise<void> {
  const existing = pending.get(url);
  if (existing) return existing;   // never fetch the same script twice
  …
}
```

The `pending` map is **promise caching**. If two files that need the CAD kernel are opened in quick succession, the second call gets the same in-flight promise instead of starting a second 8 MB download. Note also that a rejected promise is deleted from the map, so a failed download can be retried rather than being cached as a permanent failure.

The versions are pinned (`@0.0.23`) rather than floating. An unpinned CDN URL means a third party can change your application's behaviour without you deploying anything.

CHAPTER 10

The 3D engine

`src/viewer/` is plain TypeScript classes. If you have never written a class, this is a good place to learn why they exist: the engine has a long life, a lot of internal state, and a clear public interface.

10.1 `Engine.ts` — the orchestrator

One class, roughly 700 lines, owning the renderer, the camera, the controls and the model. Its public methods are the entire vocabulary the UI has for talking to the 3D world.

Construction

```
this.renderer = new THREE.WebGLRenderer({
  canvas,
  antialias: true,
  alpha: true,
  powerPreference: 'high-performance',
});
this.renderer.setPixelRatio(this.maxPixelRatio);
this.renderer.outputColorSpace = THREE.SRGBColorSpace;
this.renderer.toneMapping = THREE.ACESFilmicToneMapping;
this.renderer.shadowMap.enabled = true;
```

`antialias`	Smooths jagged edges. Costs a little performance, worth it.
`alpha`	Lets the canvas be transparent, so the CSS gradient behind it shows through.
`setPixelRatio`	Renders at the display's real density. Capped at 2 — a 3× phone screen would triple the pixel count for no visible gain.
`outputColorSpace`	Tells three.js to convert its internal linear colour to sRGB for display. Get this wrong and everything looks washed out.
`toneMapping`	Maps a wide brightness range into what a screen can show, the way a camera does. Without it, bright highlights clip to flat white.

The render loop

```
private animate() {
  if (this.disposed) return;
  this.raf = requestAnimationFrame(this.animate);
  if (this.contextLost || this.paused) return;

  if (this.controls.update(this.clock.getDelta())) {
    this.needsRender = true;
    this.lastMotion = performance.now();
  }

  this.sample();
  if (!this.needsRender) return;   // nothing changed — draw nothing
  this.needsRender = false;
  this.renderFrame();
  this.rendered++;
}
```

This is **on-demand rendering** and it is the single most valuable performance idea in the file. Most three.js examples redraw sixty times a second forever. Caliper draws only when something changed.

Every method that alters the picture calls `invalidate()`, which sets a flag. A model sitting still on screen costs nothing — no GPU work, no battery drain, no fan.

The quality governor

```
const busy = this.rendered > 0 && now - this.lastMotion < 1200;

if (busy && fps < 30 && this.quality > 0.55) {
  if (++this.slowSamples >= 2) this.setQuality(this.quality - 0.2);
} else if (busy && fps > 55 && this.quality < 1) {
  if (++this.fastSamples >= 4) this.setQuality(this.quality + 0.2);
}
```

If the frame rate drops under load, the renderer lowers its resolution rather than stuttering. When there is headroom it climbs back. Two details make it stable: it only judges performance while the camera is *moving* (an idle viewer renders nothing and would otherwise look like 0 fps), and it requires several consecutive slow samples before acting, so one hitch does not trigger it. Recovery is deliberately slower than degradation — four samples up, two down — to avoid oscillating.

Render modes without wrecking the stage

```
private assignOverride(material: THREE.Material | null) {
  this.modelGroup.traverse((node) => {
    const mesh = node as THREE.Mesh;
    if (!mesh.isMesh) return;
    if (material) {
      if (!holder.userData.baseMaterial) holder.userData.baseMaterial = mesh.material;
      mesh.material = material;
    } else if (holder.userData.baseMaterial) {
      mesh.material = holder.userData.baseMaterial;
      delete holder.userData.baseMaterial;
    }
  });
}
```

A REAL BUG THIS REPLACED

three.js offers `scene.overrideMaterial`, which looks like exactly the right tool. It is not: it repaints *everything in the scene*, including the build plate, the grid and the selection marker. "Show me the wireframe" turned the entire stage into wireframe and the model lost its floor. Swapping materials on the model's own meshes — and remembering the originals in `userData` so they can be restored — is more code and the only correct version.

Picking — turning a click into an object

```
private rayFrom(event: PointerEvent) {
  const rect = this.canvas.getBoundingClientRect();
  this.pointer.x =  ((event.clientX - rect.left) / rect.width)  * 2 - 1;
  this.pointer.y = -((event.clientY - rect.top)  / rect.height) * 2 + 1;
  this.raycaster.setFromCamera(this.pointer, this.camera);
  …
}
```

A click is a 2D point; the model is 3D. **Raycasting** bridges them: fire an imaginary ray from the camera through that pixel and find what it hits.

The arithmetic converts pixel coordinates into **normalised device coordinates** — a square from −1 to +1 in both axes, with the origin at the centre. The Y axis is negated because screen coordinates grow downward while 3D coordinates grow upward. That sign flip is the single most common bug in hand-written picking code.

Distinguishing a click from a drag

```
this.canvas.addEventListener('pointerup', (event) => {
  const moved = Math.hypot(event.clientX - down.x, event.clientY - down.y);
  if (moved > 5) return;   // an orbit, not a click
  …
});
```

Without this, every time you finished orbiting the model you would also select whatever was under the cursor. Five pixels of tolerance separates intent from noise — nobody drags four pixels on purpose, and nobody holds perfectly still while clicking.

Recovering from a lost GPU context

```
this.canvas.addEventListener('webglcontextlost', (event) => {
  event.preventDefault();          // without this, it is never restored
  this.contextLost = true;
  this.hooks.onContextChange(true);
});
```

Browsers take WebGL contexts away — a driver reset, a laptop waking from sleep, too many canvases in other tabs. Most applications ignore this and show a permanently black rectangle. Calling `preventDefault()` is what tells the browser you intend to recover.

10.2 `Stage.ts` — everything that is not the model

The build plate, its grid, the shadow catcher and three lights. Separating this from `Engine` keeps both readable: `Stage` owns the room, `Engine` owns the model and the camera.

The lighting is a classic **three-point rig**: a bright key light casting the shadow, a dim cool fill opposite it to stop shadows going black, and a rim light behind to separate the silhouette from the background. It is the same arrangement a photographer uses.

One subtlety worth stealing: the plate uses `MeshBasicMaterial`, which ignores lighting entirely. A lit plate would catch the key light's hotspot and read as another surface of the model. Unlit, it stays an even field.

10.3 `Annotations.ts` — drawing the measurements

Dimension lines drawn the way they are drawn on a paper engineering drawing: a span between two points, short witness ticks standing off each end, and the number set clear of the geometry.

It uses `LineSegments2` rather than plain lines. A standard WebGL line is locked to one device pixel wide — on a high-density display that is half a CSS pixel, too faint to trust a number to. `LineSegments2` builds each line out of triangles so it can have real screen-space width, at the cost of needing to be told the canvas size whenever it changes.

```
const view = camera.getWorldPosition(new THREE.Vector3());
if (!this.dirty && view.distanceToSquared(this.lastView) < 1e-8) return;
```

The witness ticks turn to face the camera, so the geometry is rebuilt when the view moves — but *only* then. `distanceToSquared` avoids a square root; when you only need to compare distances, never compute the root.

10.4 `Bracket.ts` — the selection marker

A faint wire cage around the selected part with solid brackets at its eight corners. The design history is instructive: a full wire box was too loud and competed with the measurements, corners alone read as debris hanging in space, and the cage is what ties them together — barely visible, but enough that the brackets are obviously the corners *of something*.

10.5 `analyze.ts`

Three pure functions: `computeStats` counts triangles, vertices, materials and textures; `computeDimensions` returns the bounding box; `disposeObject` frees GPU memory.

```
export function disposeObject(root: THREE.Object3D): void {
  root.traverse((node) => {
    const mesh = node as THREE.Mesh;
    mesh.geometry?.dispose?.();
    for (const m of list) {
      for (const value of Object.values(m)) {
        if (value?.isTexture) value.dispose();
      }
      m.dispose();
    }
  });
}
```

GARBAGE COLLECTION DOES NOT FREE GPU MEMORY

JavaScript's collector reclaims JavaScript objects. It knows nothing about the buffers and textures those objects uploaded to the graphics card. Open five models without disposing and you have leaked five models' worth of video memory. Every three.js application needs a function like this, and most of them do not have one.

CHAPTER 11

The loaders

Seven modules behind one interface. This folder is a small lesson in designing for extension.

11.1 The contract

```ts
export interface LoadResult {
  object: THREE.Object3D;
  hasAuthoredMaterials: boolean;
}

export interface LoadContext {
  files: Map<string, File>;   // every dropped file, by lower-case name
  urls:  Map<string, string>; // a blob URL for each
  onProgress: (fraction: number | null, note?: string) => void;
}
```

Every loader takes a `File` and a `LoadContext` and returns a `LoadResult`. Nothing else in the application knows or cares which one ran. Adding a new format means writing one function and adding one `case` — no other file needs to change.

11.2 `index.ts` — the dispatcher

```ts
export function pickPrimary(files: File[]): File | null {
  const candidates = files.filter(
    (f) => isSupported(f.name) && !COMPANION_EXTS.has(extOf(f.name)),
  );
  if (!candidates.length) return null;
  return candidates.sort((a, b) => b.size - a.size)[0];
}
```

Drop a folder and you may get thirty files. Which one is the model? A `.mtl` beside an `.obj` is payload, not a subject — so companion extensions are filtered out. When two real models remain, the larger wins, because that is almost always the one you meant.

Resolving sibling files without a server

```ts
function managerFor(ctx: LoadContext): THREE.LoadingManager {
  const manager = new THREE.LoadingManager();
  manager.setURLModifier((url) => {
    if (url.startsWith('blob:') || url.startsWith('data:')) return url;
    const clean = decodeURIComponent(url.split('?')[0].split('#')[0]);
    return ctx.urls.get(clean.toLowerCase())
        ?? ctx.urls.get(baseOf(clean).toLowerCase())
        ?? url;
  });
  return manager;
}
```

A glTF file says "load `textures/wood.png`". There is no server to serve that. The URL modifier intercepts every request three.js makes and swaps in the blob URL of the matching dropped file. This is an **interception pattern**, and it is worth recognising — you will meet it again in service workers and test mocks.

11.3 The individual loaders

FILE	HANDLES	NOTABLE
`mesh.ts`	Twelve mesh formats	Mostly a `switch` delegating to three.js's own loaders. STL gets extra work: it stores loose triangles with no shared vertices, so `mergeVertices` welds them, which is what makes smooth shading and clean edges possible.
`occt.ts`	STEP, IGES, BREP, FCStd	Downloads an OpenCascade kernel compiled to WebAssembly and tessellates the surfaces. FCStd is unzipped first with `fflate`.
`ifc.ts`	IFC building models	Streams meshes one building element at a time. Element names are read only for models under 4,000 elements, because it is one query each.
`rhino.ts`	Rhinoceros `.3dm`	Uses the official `rhino3dm` WebAssembly build.
`dotbim.ts`	`.bim`	A small JSON schema. Instances share geometry, so one mesh reused a thousand times stays one buffer in memory.
`off.ts`	`.off`	Hand-written parser — three.js has none. A readable example of tokenising a text format.

Progress that does not lie

```
ctx.onProgress(null, 'Downloading the CAD kernel (one time, ~8 MB)');
…
ctx.onProgress(i / shapes.length, `Tessellating shape ${i + 1} of ${shapes.length}`);
```

`null` means "working, but I cannot say how far" and drives an indeterminate bar. A number drives a real one. Never fake a percentage — a bar that sticks at 90% is worse than one that admits it does not know.

11.4 A defensive detail worth copying

```
shapes.forEach((entry, i) => {
  try {
    root.add(toObject(occt.ReadBrepFile(zip[entry], PARAMS), …));
  } catch {
    /* One unreadable shape should not sink the whole document. */
  }
});

if (!root.children.length) throw new Error('None of the shapes could be read.');
```

A FreeCAD document may hold a hundred shapes. If one is corrupt, opening ninety-nine is far better than opening none — but if *all* fail, that must still be an error rather than a silent empty scene. Partial success is a real state, and deciding how to handle it is part of the design.

CHAPTER 12

The stylesheets

Three files, loaded in order, each with a different scope. No CSS framework, no CSS-in-JS — just custom properties and class names.

12.1 The three layers

`tokens.css`	Custom properties only. Colour, type, spacing, timing. No selectors that style anything.
`base.css`	Reset, document defaults, the page grid, scrollbars, focus rings, reduced-motion handling.
`ui.css`	Every component's appearance.

Order matters: they are imported in that sequence in `main.tsx`, so tokens exist before anything uses them and base rules can be overridden by component rules without `!important`.

12.2 Design tokens

```
:root {
  --axis-x: #ff5f56;
  --axis-y: #57d364;
  --axis-z: #4aa3ff;

  --accent:      #ffb224;
  --accent-ink:  #1a1206;   /* text that sits ON the accent */
  --accent-wash: rgba(255, 178, 36, 0.14);
  …
}

:root[data-theme='light'] {
  --accent:     #a85c00;
  --accent-ink: #ffffff;
  …
}
```

A **design token** is a named decision. Because the light theme only redefines the variables, not the rules that use them, theming costs nothing at the component level — no `.dark` variants anywhere.

`--accent-ink` is the subtle one: on the dark theme the accent is a bright amber that needs *dark* text on it; on the light theme it is a bronze that needs white. If the ink colour were hardcoded, one of the two themes would fail contrast. Any time a colour pairs with another colour, both halves of the pair should be tokens.

The colour system

Three families that never overlap:

Amber	Authorship — selection, measurements, primary buttons. Anything amber is something you put there.
The axis triad	Position — the gizmo, the size rows, the per-axis split of a distance.
Grey	Everything else.

The choice of amber is not arbitrary. A 3D viewer has already spent red, green and blue on the X/Y/Z convention, which is not negotiable because every other tool in the field uses it. A green brand colour has to fight axis-Y for the same wavelength, and one of them loses. Amber is the one hue that cannot collide. **Constraints from the domain should drive the palette** — that is what makes a design feel like it belongs to its subject rather than being applied on top of it.

12.3 Layout with CSS Grid

```
.shell {
  display: grid;
  grid-template-rows: var(--rail-top) 1fr var(--strip);
  grid-template-columns: var(--rail-side) 1fr auto;
  grid-template-areas:
    'top    top    top'
    'rail   stage  panel'
    'strip  strip  strip';
  height: 100dvh;
}

.shell[data-panel='closed'] { grid-template-columns: var(--rail-side) 1fr 0; }
```

Named grid areas make the layout readable as a picture in the source. Collapsing the panel is one line changing one column to zero — no JavaScript measuring anything.

`100dvh` is the *dynamic* viewport height. On mobile, `100vh` famously includes the area behind the browser's address bar, so a full-height layout overflows. `dvh` tracks the space actually available.

12.4 Styling from state, not classes

```
.tool[aria-pressed='true'] {
  color: var(--accent);
  background: var(--accent-wash);
}
```

The active tool is styled from `aria-pressed` — the same attribute that tells a screen reader the button is a toggle and currently on. One source of truth serving both appearance and accessibility, instead of a `.is-active` class that can drift out of sync with the ARIA state.

12.5 Respecting reduced motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.001ms !important;
    transition-duration: 0.001ms !important;
  }
}
```

Some people get motion sickness from interface animation, and the operating system lets them say so. Four lines honour that across the entire application. This is the cheapest accessibility win in frontend development and it is skipped almost everywhere.

PART III

The ideas behind the code

React patterns, state management, 3D graphics and the measuring mathematics — explained from first principles, using this application as the worked example. Read these in order.

CHAPTER 13

React, as used here

React's whole idea is one sentence: **you describe what the screen should look like for a given state, and React works out the DOM changes.** Everything else is detail.

13.1 Components and JSX

```tsx
function Row({ label, value }: { label: string; value: string }) {
  return (
    <div className="field">
      <span className="field-label">{label}</span>
      <span className="field-value">{value}</span>
    </div>
  );
}
```

A component is a function that returns markup. JSX is syntax sugar — the compiler turns those tags into function calls. Two consequences: `className` instead of `class` (because `class` is a reserved word), and `{}` to drop back into JavaScript.

Props are the function's arguments. They flow **down** only. A child can never write to its parent's props — if it needs to cause a change, the parent passes down a callback.

13.2 The hooks this project uses

HOOK	WHAT IT IS FOR	USED IN
`useState`	A value that, when changed, re-renders the component.	`useMediaQuery`, `useFileDrop`
`useEffect`	Run code after render; clean up when dependencies change or the component unmounts.	`Viewport`, `Gizmo`, `App`
`useRef`	A mutable box that survives renders and does *not* trigger them.	DOM nodes, the drag counter, the animation frame id
`useCallback`	Keep a function's identity stable between renders.	`App`, `useFileDrop`
`useMemo`	Keep a computed value's identity stable between renders.	`App`

`useState`

```tsx
const [dragging, setDragging] = useState(false);
```

Array destructuring gives you the current value and a setter. Calling the setter schedules a re-render. Two rules that catch everyone once:

- **State updates are not immediate.** Reading the variable right after setting it gives the old value — the new one arrives on the next render.

- **Never mutate state.** `list.push(x); setList(list)` does nothing, because it is the same array reference and React compares by identity. Write `setList([...list, x])`.

`useEffect` and its dependency array

```
useEffect(() => {
  // … set something up …
  return () => { /* … tear it down … */ };
}, [attach, detach]);
```

The array at the end is the contract: re-run this effect whenever one of these values changes. Three shapes:

`[]`	Run once on mount, clean up on unmount.
`[a, b]`	Run on mount and whenever `a` or `b` changes.
omitted	Run after *every* render. Almost always a mistake.

The cleanup function is the half people forget. If your effect adds an event listener, starts a timer, opens a connection or creates a 3D engine, the cleanup must undo it. Otherwise you leak — and in `StrictMode`, React deliberately mounts, unmounts and remounts every component in development specifically to make that leak visible.

`useRef` versus `useState`

Both hold a value across renders. The difference is that changing a ref does not re-render. Use a ref when the value is not displayed:

```
const canvasRef = useRef<HTMLCanvasElement>(null);   // a DOM node
const depth     = useRef(0);                         // a running count
const raf       = useRef(0);                         // an animation frame id
```

In `useFileDrop`, `dragging` is state because it shows an overlay; `depth` is a ref because it is bookkeeping. Getting that split right is most of what makes a component efficient.

Why `useCallback` and `useMemo` exist

```
useHotkeys(useMemo(() => ({ open: openPicker, screenshot }), [openPicker, screenshot]));
```

Every render creates new function and object values. If one of those is in an effect's dependency array, the effect re-runs every render — here, that would tear down and re-register the keyboard listener sixty times a second. `useMemo` and `useCallback` keep the identity stable so the dependency comparison passes.

> **DO NOT REACH FOR THESE BY DEFAULT**
>
> They are not free — they cost memory and make code harder to read. Use them when a value feeds a dependency array or a memoised child, not "for performance" in general. Premature memoisation is as much of a smell as premature optimisation anywhere else.

13.3 The rules of hooks

Two rules, and they are absolute:

1. Only call hooks at the top level of a component or another hook. Never inside a condition, loop or nested function.
2. Only call them from React functions.

The reason is mechanical: React tracks hooks by *call order*. If a condition skips one, every subsequent hook shifts by one slot and reads another hook's state. This is why `Panel.tsx` calls all of its `useViewer` hooks before the `if (!model) return`, not after.

13.4 Keys in lists

```
{measurements.map((measurement) => (
  <li key={measurement.id} className="measure"> … </li>
))}
```

A key tells React which item is which across renders, so it can move elements instead of rebuilding them. Use a stable id from your data. Using the array index is a bug waiting to happen: delete the first measurement and every subsequent key shifts, so React reuses the wrong DOM node and any internal state — focus, scroll position, an animation midway — lands on the wrong row.

CHAPTER 14

Zustand, from scratch

Zustand is about a kilobyte. Understanding what it actually does removes most of the mystery from state management generally.

14.1 What a store really is

Strip the library away and a store is three things: a value, a list of functions to call when it changes, and a way to change it.

```
// A minimal store, written by hand.
function createStore(initial) {
  let state = initial;
  const listeners = new Set();

  return {
    getState: () => state,
    setState: (partial) => {
      state = { ...state, ...partial };
      listeners.forEach((l) => l());
    },
    subscribe: (listener) => {
      listeners.add(listener);
      return () => listeners.delete(listener);
    },
  };
}
```

That is the whole concept. Zustand adds a React hook that subscribes on mount, unsubscribes on unmount, and — crucially — only re-renders when the *slice you selected* changed.

14.2 Creating the store

```
export const useViewer = create<ViewerState>((set, get) => ({
  model: null,
  unit: 'mm',

  setUnit(unit) {
    set({ unit });
  },

  frameSelection() {
    const { engine, selection } = get();   // read current state
    if (engine && selection) engine.frameSelection(selection.id);
  },
}));
```

`set` writes, `get` reads the current value without subscribing. Actions live beside data, so a component imports one thing.

14.3 Selectors and re-renders

```
const model = useViewer((s) => s.model);
```

After every `set`, Zustand runs each subscriber's selector and compares the result to the previous one with `Object.is`. Same result, no re-render.

> **THE OBJECT-SELECTOR TRAP**
>
> `useViewer((s) => ({ a: s.a, b: s.b }))` creates a *new object* every time, so the identity check always fails and the component re-renders on every store change. Either select the two values separately, or pass a shallow-equality comparator. This project selects separately, which is simpler and always correct.

14.4 Using the store outside React

```
const store = useViewer.getState();
const engine = new Engine(canvas, { onFrame: store.setFrame, … });
```

This is how the 3D engine talks to the UI without importing React. It is handed plain functions from the store and calls them when something happens. `getState()` works anywhere — event handlers, timers, class methods — because the store lives outside the component tree.

14.5 When you need a store at all

Not always. Reach for one when state is genuinely shared across distant parts of the tree, or when it must be readable from non-React code. A dropdown's open/closed state belongs in `useState` in the dropdown, not in a global store — putting local state in a global store is as much of a mistake as prop-drilling shared state.

CHAPTER 15

three.js, from scratch

Every 3D scene, in any engine, is the same five nouns: a scene, a camera, some meshes, some lights and a renderer.

15.1 The five nouns

Scene	The container. A tree of objects, each with a position, rotation and scale.
Camera	The point of view. A perspective camera has a field of view and near/far clipping distances.
Mesh	A visible object = **geometry** (the shape) + **material** (how the surface responds to light).
Light	Illumination. Without one, most materials render black.
Renderer	Takes a scene and a camera and draws a frame onto a canvas.

```
const scene    = new THREE.Scene();
const camera   = new THREE.PerspectiveCamera(42, width / height, 0.01, 4000);
const renderer = new THREE.WebGLRenderer({ canvas });

scene.add(new THREE.Mesh(geometry, material));
renderer.render(scene, camera);
```

Those four arguments to the camera are field of view in degrees, aspect ratio, and the near and far planes. Anything closer than near or further than far is not drawn. Setting near to a very small number causes **z-fighting** — surfaces flickering against each other — because depth precision is spent on empty space. Caliper computes both from the model's size for exactly this reason.

15.2 Geometry is just arrays

```
const geometry = new THREE.BufferGeometry();
geometry.setAttribute('position', new THREE.Float32BufferAttribute(positions, 3));
geometry.setIndex(indices);
geometry.computeVertexNormals();
```

A shape is a flat array of numbers. `positions` holds `x, y, z, x, y, z, …` and the `3` says how many numbers make one vertex.

The **index** is an optimisation with real consequences. A cube has 8 corners but 12 triangles = 36 vertex slots. Storing 8 positions and 36 indices is smaller and, more importantly, tells the GPU that triangles *share* corners — which is what lets normals be averaged into smooth shading.

A **normal** is a vector pointing away from the surface at each vertex. It is how a material knows how much light a point receives. `computeVertexNormals()` calculates them from the triangles when the file did not supply them.

WHY STL GETS SPECIAL TREATMENT

STL stores every triangle independently, with no shared vertices at all — a cube arrives as 36 unconnected points. `mergeVertices` welds coincident ones back together, which is what makes both smooth shading and clean edge outlines possible. It is one line in `mesh.ts` and it is the difference between a faceted blob and a model.

15.3 Materials

`MeshStandardMaterial`	Physically based. Has `metalness` and `roughness`, responds to lights and to the environment. The default for models here.
`MeshBasicMaterial`	Ignores light entirely — a flat colour. Used for the build plate, so it does not catch the key light's hotspot.
`LineBasicMaterial`	For lines. Always one device pixel wide, whatever you set.
`ShadowMaterial`	Invisible except where a shadow falls. The trick that lets a shadow land on nothing.

15.4 The scene graph and transforms

```
const holder = new THREE.Group();
holder.add(object);

holder.scale.setScalar(scale);
holder.position.set(-center.x * scale, -box.min.y * scale, -center.z * scale);
```

Transforms are inherited: move a group and everything inside moves with it. Caliper uses this deliberately — the loaded model keeps whatever transform its file gave it, and all the framing happens on a wrapper group. A file that places its geometry a kilometre from the origin still lands centred on the plate, and its own coordinates are left untouched so measurements can be converted back.

```
private toModel(world: THREE.Vector3): THREE.Vector3 {
  if (!this.holder) return world.clone();
  return this.holder.worldToLocal(world.clone());
}
```

`worldToLocal` is the inverse of that wrapper transform. It is how a click in viewer space becomes a number in the file's own units.

15.5 Raycasting

```
raycaster.setFromCamera(pointer, camera);
const hits = raycaster.intersectObjects(modelGroup.children, true);
```

Fire a ray from the camera through a normalised screen point and test it against every triangle. The results come back sorted nearest-first, each with the object, the exact 3D `point`, the `distance`, and the `face` that was hit. That `face` — its three vertex indices — is what the snapping in the next chapter is built on.

15.6 Disposal

```
mesh.geometry.dispose();
material.dispose();
texture.dispose();
renderer.dispose();
```

Say it again because it is the most common three.js bug: JavaScript's garbage collector does not free GPU memory. Every geometry, material, texture and render target you create must be disposed explicitly.

CHAPTER 16

The measuring mathematics

The core feature, and the most interesting maths in the project. All of it lives in `src/viewer/measure.ts`.

16.1 The problem with a raw hit

A raycast returns the exact point where the ray met a triangle. That point is almost never what a person meant. They were aiming at a corner, and they hit three pixels away from it. A measurement that is "nearly" between two corners is not a measurement.

16.2 Snapping

The insight: every triangle offers seven better candidates than the raw hit — three corners, three edge midpoints, and the centroid.

```
const candidates: Candidate[] = [
  { point: a, kind: 'vertex',   rank: 0 },
  { point: b, kind: 'vertex',   rank: 0 },
  { point: c, kind: 'vertex',   rank: 0 },
  { point: a.clone().add(b).multiplyScalar(0.5), kind: 'midpoint', rank: 1 },
  { point: b.clone().add(c).multiplyScalar(0.5), kind: 'midpoint', rank: 1 },
  { point: c.clone().add(a).multiplyScalar(0.5), kind: 'midpoint', rank: 1 },
  { point: a.clone().add(b).add(c).divideScalar(3),  kind: 'centre',  rank: 2 },
];
```

A midpoint is the average of two corners; a centroid is the average of three. Both fall straight out of vector addition.

Measuring "close" in the right space

```
for (const candidate of candidates) {
  const screen   = toScreen(candidate.point, camera, width, height);
  const distance = Math.hypot(screen.x - cursor.x, screen.y - cursor.y);
  if (distance > bestDistance) continue;
  if (best && distance === bestDistance && candidate.rank >= best.rank) continue;
  best = candidate;
  bestDistance = distance;
}
```

> **THE KEY DESIGN DECISION**
>
> Distance is measured **on screen, in pixels** — not in 3D. If it were measured in 3D, the snap radius would feel enormous when zoomed out and vanish when zoomed in. In screen space, 14 pixels is 14 pixels at any zoom, which is what the hand is actually doing. Whenever an interaction competes with a camera, ask which space the user is working in.

The `rank` breaks ties toward the more deliberate target: a corner beats a midpoint beats a centroid at equal distance, because a corner is what someone aims at.

Projecting 3D to 2D

```
function toScreen(world, camera, width, height) {
  _projected.copy(world).project(camera);
  return {
    x: (_projected.x * 0.5 + 0.5) * width,
    y: (-_projected.y * 0.5 + 0.5) * height,
  };
}
```

`project()` applies the camera's matrices to give normalised device coordinates in the range −1…+1. The arithmetic remaps that to pixels, flipping Y because screens count downward.

Note `_projected` is a module-level vector reused on every call. This runs for seven candidates on every pointer move — allocating seven new vectors each time would create constant garbage-collection pressure. Reusing scratch objects in hot paths is standard practice in graphics code.

16.3 Distance

```
const delta = model[1].clone().sub(model[0]);
if (delta.lengthSq() < COINCIDENT) return null;
return {
  value: delta.length(),
  delta: [Math.abs(delta.x), Math.abs(delta.y), Math.abs(delta.z)],
};
```

Straight Pythagoras in three dimensions. The per-axis split is often more useful than the diagonal — "how far apart are these holes horizontally" is a more common question than the true 3D span.

`lengthSq()` again avoids a square root for a comparison. And the conversion to model space happens *before* the subtraction, so the number is in the file's units rather than the viewer's.

16.4 Fitting a circle through three points

The diameter tool takes three points on an arc and finds the circle through them. In 2D this is the classic circumcircle; in 3D the three points define a plane and the problem reduces to the same thing.

```
export function circleThrough(a, b, c) {
  const ab = b.clone().sub(a);
  const ac = c.clone().sub(a);
  const normal = ab.clone().cross(ac);

  const denominator = 2 * normal.lengthSq();
  if (denominator < 1e-18) return null;      // collinear — no circle exists

  const toCentre = normal.clone().cross(ab).multiplyScalar(ac.lengthSq())
    .add(ac.clone().cross(normal).multiplyScalar(ab.lengthSq()))
    .divideScalar(denominator);

  return { centre: a.clone().add(toCentre), radius: toCentre.length() };
}
```

Two vector operations do the work.

The cross product of two vectors gives a third perpendicular to both. Its *length* equals the area of the parallelogram they span — which is zero exactly when they are parallel. That is why `normal.lengthSq()` being near zero is the collinearity test: if the three points lie on a line, no circle passes through them, and the function returns `null` instead of dividing by zero and producing `Infinity`.

The formula itself is the standard circumcentre in vector form. You do not need to derive it, but you should recognise the shape of the guard around it. Every geometric formula has a degenerate case, and handling it explicitly — rather than letting a `NaN` propagate into the interface — is the difference between code that works and code that works on your test file.

When it returns `null`, the engine reports why:

```
diameter: 'Those three points sit in a straight line, so no circle passes ' +
          'through them. Spread them around the arc.',
```

An error message that says what to do next is worth ten that say what went wrong.

16.5 World space versus model space

Every measurement stores both.

```
export interface MeasurePoint {
  world: [number, number, number];   // what the annotation is drawn from
  model: [number, number, number];   // what the readout shows
  kind: SnapKind;
}
```

The engine scales every model to a fixed 8-unit diagonal so lighting and the grid behave the same for a bolt and a building. Drawing must happen in that space; the number a person reads must not. Keeping both, rather than converting on the fly, means the annotation never drifts from its label.

CHAPTER 17

Routing — and why this app has none

You asked about routing. The honest answer is that Caliper has no router, and understanding *why* teaches more than adding one would.

17.1 What routing is

Routing maps a URL to a screen. In a traditional website the server does this: `/about` returns the about page. In a single-page application the server returns the same HTML for every path, and JavaScript decides what to render based on `window.location`.

17.2 Why this app does not need it

Caliper has exactly one screen. There is no About page, no settings page, no list-then-detail flow. What looks like navigation — the panel opening, the measure tool activating — is state, not location. Nobody wants to bookmark "the measure tool is selected", and nobody expects the back button to undo it.

> **THE TEST FOR WHETHER SOMETHING BELONGS IN THE URL**
>
> Should this survive a refresh? Should it be shareable as a link? Should the back button undo it? Three yeses means it is a route. In Caliper only one thing passes: *which model is open* — and that is handled by a query parameter without any router at all.

```
// The entire routing layer of this application.
const params = new URLSearchParams(window.location.search);
const raw    = params.get('model');
```

Adding React Router for that would mean roughly 12 kB of JavaScript, a new set of concepts, and a wrapper around the whole tree — to read one query string the platform already parses for you.

17.3 What routing would look like if you added it

Suppose version 2 gains a gallery of saved models, a settings page and a shareable per-model view. Now three URLs matter and a router earns its place.

```
npm install react-router-dom
```

```
import { createBrowserRouter, RouterProvider, useParams, Link } from 'react-router-dom';

const router = createBrowserRouter([
  { path: '/',             element: <Viewer /> },
  { path: '/gallery',      element: <Gallery /> },
  { path: '/model/:id',    element: <Viewer /> },
  { path: '*',             element: <NotFound /> },
]);

function App() {
  return <RouterProvider router={router} />;
}
```

Inside a route, the dynamic segment is read with a hook:

```
function Viewer() {
  const { id } = useParams();          // '/model/abc' -> id === 'abc'
  …
}
```

And you navigate with `Link` rather than `a`, so the browser does not reload the page:

```
<Link to="/gallery">Gallery</Link>
```

The one server-side requirement

If someone loads `/gallery` directly, the server must return `index.html` rather than a 404 — because that file does not exist on disk. Caliper already has this configured, which is why the rewrite is in `vercel.json` despite there being no router yet:

```
"rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
```

This is the single most common deployment bug in single-page applications: it works locally, works when you click through, and 404s when someone shares a deep link. Vercel checks the filesystem before applying rewrites, so real files like `/assets/index.js` still serve correctly.

17.4 The lesson

Do not install a router because React apps have routers. Install one when you have more than one URL that matters. The same reasoning applies to state libraries, component libraries and everything else: **let the problem ask for the tool.**

CHAPTER 18

Making it fast

Seven techniques, each responding to a measured problem rather than a guess.

18.1 On-demand rendering

Draw only when something changed. A still model costs zero GPU work. Covered in 10.1 — it is the largest single win in the project.

18.2 Adaptive resolution

When the frame rate drops under load, lower the device pixel ratio instead of stuttering. A slightly softer image that responds instantly beats a sharp one that lags.

18.3 Bypassing React for high-frequency updates

Measurement tags and the orientation gizmo write to the DOM directly. Sixty state updates a second would re-render the entire interface sixty times a second.

18.4 The `backdrop-filter` discovery

A REAL 27× REGRESSION, FOUND BY MEASURING

The command palette originally had a full-screen scrim with `backdrop-filter: blur(3px)`. Frames took **331 ms** — three per second. Blurring a full-viewport area that contains a live WebGL canvas forces the compositor to read back and re-blur the entire drawing buffer every frame.

Removing it, and pausing the render loop behind modal overlays, took frame time to **12 ms**. The lesson is not "never use backdrop-filter" — small chips still use it here. It is that expensive CSS effects over live-updating content are a different cost class, and that you find this by measuring, not by reading.

18.5 Throttling with `requestAnimationFrame`

```
this.canvas.addEventListener('pointermove', (event) => {
  if (this.hoverQueued) return;
  this.hoverQueued = true;
  requestAnimationFrame(() => {
    this.hoverQueued = false;
    …
  });
});
```

Pointer events can fire faster than the display refreshes. Raycasting on every one is wasted work. This runs at most one per frame — the cheapest correct throttle there is, and it is self-synchronising with the display.

18.6 Budgets

```
const EDGE_BUDGET = 350_000;
const HOVER_LIMIT = 6000;
```

Edge outlining stops after 350,000 triangles; hover picking switches off above 6,000 parts. A feature that makes a large model unusable is worse than a feature that politely stops. Note that the truncation is *reported* to the user rather than hidden.

18.7 Code splitting

```
manualChunks: {
  three: ['three'],
  react: ['react', 'react-dom'],
}
```

Vendor libraries go into separate bundles with their own content hashes. When your application code changes, visitors re-download only that chunk — React and three.js stay in their browser cache across deploys.

18.8 Lazy WebAssembly

The 5–12 MB CAD and BIM kernels are fetched on first use. Someone who only opens STL files never downloads them at all.

BUNDLE	RAW	GZIPPED
CSS	24.9 kB	6.0 kB
React chunk	140.9 kB	45.3 kB
three.js chunk	564.0 kB	144.3 kB
Application	592.7 kB	180.6 kB

Gzipped size is what actually travels. Roughly 376 kB total, of which the 3D engine and its loaders are the overwhelming majority — which is exactly what you would expect from a 3D application, and a useful reminder to check that your bundle's shape matches your app's purpose.

CHAPTER 19

TypeScript in this project

TypeScript produces no runtime code. It is a checker that runs before your code ships, and its value is proportional to how honestly you describe your data.

19.1 Union types as a state machine

```ts
export type Status = 'idle' | 'loading' | 'ready';
export type MeasureTool = 'off' | 'distance' | 'diameter';
export type RenderMode = 'shaded' | 'wireframe' | 'edges';
```

These are **string literal unions** and they are the most useful feature in everyday TypeScript. The compiler knows the complete set of values, so a typo is an error, autocomplete works, and a `switch` that misses a case can be caught.

Compare with `status: string`, which permits `'redy'` and gives you nothing.

19.2 Deriving types from other types

```ts
const POINTS_NEEDED: Record<Exclude<MeasureTool, 'off'>, number> = {
  distance: 2,
  diameter: 3,
};
```

`Exclude<MeasureTool, 'off'>` is `'distance' | 'diameter'`. `Record<K, V>` builds an object type with those keys. Together they mean: **if you add a third tool, this object stops compiling until you handle it.** The type system becomes a to-do list that cannot be ignored.

This pattern — deriving a lookup table's keys from a union — is one of the highest-value things you can learn in TypeScript. It converts "I hope I remembered to update everywhere" into a compiler error.

19.3 Generic functions

```ts
setDisplay<K extends keyof Display>(key: K, value: Display[K]): void;
```

This says: the second argument's type depends on the first. `setDisplay('grid', true)` compiles; `setDisplay('grid', 'yes')` does not, because `Display['grid']` is `boolean`. One signature covering every property, with full checking on each.

19.4 The strict flags that matter

```
{
  "strict": true,
  "noUnusedLocals": true,
  "noUnusedParameters": true,
  "noFallthroughCasesInSwitch": true
}
```

`strict`	Turns on every safety check, most importantly `strictNullChecks` — you cannot use a possibly-null value without proving it is not.
`noUnusedLocals`	Dead variables become errors. This is how the leftover fields from removed features were caught during the v1 cleanup.
`noFallthroughCasesInSwitch`	Catches a missing `break`.

Start every project with `strict: true`. Adding it later to a large codebase means hundreds of errors at once; starting with it means handling each as you write.

19.5 Escape hatches, and when they are fair

```
const mesh = node as THREE.Mesh;
if (!mesh.isMesh) return;
```

three.js's `traverse` yields `Object3D`, and there is no way for the compiler to know which are meshes. The cast is immediately followed by a runtime check, which is the honest version of this pattern: assert the type, then verify it. A cast with no check afterwards is a lie that will eventually be found out.

Note also what is *not* here: `any` appears nowhere in the source. Where a type is genuinely unknown, the code writes the shape it expects — see the hand-written interfaces at the top of `ifc.ts` and `dotbim.ts` describing third-party data.

CHAPTER 20

Accessibility

Not a checklist bolted on at the end. Six habits, each cheap, that were applied while the components were written.

20.1 Semantic elements

`<header>`, `<main>`, `<nav>`, `<aside>`, `<footer>` are used for what they mean. Screen readers offer landmark navigation based on them, which is how a non-sighted user jumps around a page. A `div` soup offers nothing to jump to.

Every interactive element is a real `<button>`. A clickable `div` is not focusable, is not announced as a control, and does not respond to `Enter` or `Space` without you reimplementing all of it.

20.2 Labelling

```
<button className="tool" aria-pressed={active} aria-label={label}>
  {icon}
  <span className="tip">{label}<kbd className="kbd">{keys}</kbd></span>
</button>
```

An icon-only button contains no text, so it needs `aria-label`. `aria-pressed` communicates toggle state — and, as covered in 12.4, also drives the CSS, so the two cannot drift apart.

Where text must exist for screen readers but not on screen, the `.sr-only` class hides it visually while leaving it in the accessibility tree. Never use `display: none` for this — it removes the text from both.

20.3 Focus visibility

```
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}
```

Keyboard users need to see where they are. `:focus-visible` rather than `:focus` shows the ring for keyboard navigation but not after a mouse click, which is why designers used to remove focus rings entirely. Now you can have both.

20.4 Live regions

```
<div className="toasts" role="status" aria-live="polite">
```

A toast appears without any user action, so a screen reader would otherwise never mention it. `aria-live="polite"` announces it at the next natural pause; `assertive` interrupts and should be reserved for genuine emergencies.

20.5 Colour contrast

Every foreground/background pair in `tokens.css` was checked against WCAG AA — 4.5:1 for body text — in both themes. This constrained the design in a useful way: the dark theme's bright amber cannot carry white text, so the light theme uses a darker bronze that can. That inversion is the entire reason `--accent-ink` exists as a token.

20.6 Respecting preferences

Reduced motion is honoured globally (12.5). The colour scheme follows the operating system until the visitor chooses otherwise. Pinch-zoom is not blocked. These cost four lines, one media query and one omitted attribute respectively — and each of them is regularly got wrong on sites with far bigger budgets than this one.

PART IV

From repository to live site

The build tool, the type checker, git, continuous integration and deployment. This is the part most tutorials skip, and the part that turns a folder of files into something other people can use.

CHAPTER 21

Vite and the build

Vite does two quite different jobs: it serves your source directly during development, and it bundles it for production.

21.1 What happens in development

Modern browsers understand ES modules natively. Vite exploits that: it serves your files almost unchanged, transforming each one only when the browser asks for it.

The consequence is that startup time does not grow with your project. A bundler like Webpack must process everything before serving the first page; Vite processes the handful of files that page actually imports.

Dependencies are handled differently — they are pre-bundled once with esbuild (written in Go, and roughly a hundred times faster than a JavaScript bundler) and cached, because `three` alone is hundreds of small modules and serving each individually would flood the browser with requests.

Hot Module Replacement

Save a file and Vite pushes just that module to the browser, which swaps it in place. With `@vitejs/plugin-react`, component state survives — you can tweak the panel's styling without losing the model you have loaded. That preservation is the entire reason the plugin exists.

21.2 What happens in a production build

`npm run build` is `tsc -b && vite build` — two steps, and the order matters. TypeScript checks types first and stops everything if there is an error. Vite would happily bundle broken types, because it strips them without checking. Wiring the checker into the build script is what makes CI meaningful.

Then Vite, using Rollup, does five things: resolves the import graph from `index.html`, tree-shakes unreachable code, minifies, adds content hashes to filenames, and rewrites the HTML to point at the hashed files.

Content hashing and caching

```
dist/assets/index-CFHW1v1l.js
```

The hash is derived from the file's contents. Change the code and the name changes. This is what makes it safe to tell browsers to cache those files forever — a new deploy produces new names, so there is no stale-cache problem, and no cache-busting query strings.

`index.html` itself is never cached, because it is the file that points at the hashed ones.

21.3 `vite.config.ts`

```
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: { '@': fileURLToPath(new URL('./src', import.meta.url)) },
  },
  build: {
    target: 'es2022',
    chunkSizeWarningLimit: 1600,
    rollupOptions: {
      output: {
        manualChunks: { three: ['three'], react: ['react', 'react-dom'] },
      },
    },
  },
});
```

`alias`	Makes `@/store/useViewer` mean `src/store/useViewer`. Without it, a file three folders deep writes `../../../store/useViewer`, which breaks whenever a file moves. The alias must also be declared in `tsconfig.json` so the editor understands it.
`target`	How modern the output may be. `es2022` means no transpiling of features that all current browsers support — smaller, faster output at the cost of dropping very old browsers.
`manualChunks`	Splits vendor libraries into their own bundles so they cache independently of your code.

21.4 The `public/` folder

Files in `public/` are copied to the output root untouched — no hashing, no processing. Use it for things that must have a stable, known URL: `robots.txt`, `manifest.webmanifest`, `favicon.svg`, the social preview image.

Everything else — anything imported from your source — should live in `src/` so it gets hashed and optimised.

CHAPTER 22

TypeScript configuration

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "noEmit": true,
    "isolatedModules": true,
    "skipLibCheck": true,
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": { "@/*": ["src/*"] }
  },
  "include": ["src"]
}
```

`lib`	Which built-in type definitions to load. `DOM` gives you `document` and `window`; without it the compiler does not believe the browser exists.
`moduleResolution: "bundler"`	Tells TypeScript that a bundler will resolve imports, so extensionless paths and package `exports` maps work the way Vite treats them.
`jsx: "react-jsx"`	The modern transform. This is why no file needs `import React from 'react'` just to use JSX.
`noEmit: true`	TypeScript checks and produces nothing. Vite does the actual compiling. Two tools, one job each.
`isolatedModules`	Requires every file to be compilable on its own, which is what esbuild needs. It is why type-only imports are written `import type { … }`.
`skipLibCheck`	Do not type-check inside `node_modules`. A large speed win, and you cannot fix those errors anyway.
`paths`	The editor's half of the `@/` alias. Vite has its own copy in `vite.config.ts`; both are needed and they must agree.

CHAPTER 23

Version control

Git is not a backup system. It is a record of *why* the code looks the way it does, and the quality of that record is entirely up to you.

23.1 The branch model

BRANCH	RULE
`main`	What is deployed. Always working. Only receives merges from `dev`.
`dev`	Where work lands. May be broken briefly.
feature branches	One per change, merged into `dev`.

For a solo project this is close to the minimum that is still useful. The value is that `main` always represents something shippable, so "deploy the latest" is never a gamble.

```
git checkout dev
git checkout -b feature/angle-tool     # branch off dev
# … work, commit …
git push -u origin feature/angle-tool  # open a PR against dev
```

23.2 Writing a commit message

The format that has become standard: a summary line under about 70 characters in the imperative mood, a blank line, then prose explaining **why**.

```
Stop the deploy failing over a setting a workflow cannot change

Creating a GitHub Pages site through the API needs repository admin
rights that GITHUB_TOKEN does not carry, whatever `permissions` says.
That is why actions/configure-pages returns "Resource not accessible by
integration", and no amount of workflow configuration fixes it.

Failing every push over that is noise. The deploy now runs a preflight
job that asks the API whether a Pages site exists …
```

The diff already shows *what* changed. It can never show what you tried first, what you ruled out, or why the obvious approach does not work. That is what the body is for, and it is the message you will thank yourself for in eight months.

Imperative mood — "Fix the leak", not "Fixed" or "Fixes" — because a commit completes the sentence "applying this commit will…".

23.3 `.gitignore`

```
node_modules
dist
*.local
.DS_Store
*.tsbuildinfo
```

Never commit `node_modules` — it is reconstructible from `package-lock.json` and is enormous. Never commit `dist` — it is generated, and committing build output guarantees merge conflicts on files nobody edits.

23.4 `.gitattributes`

```
* text=auto eol=lf

*.png binary
*.stl binary
…
```

Windows ends lines with CRLF, everything else with LF. Without this, a file edited on Windows can appear entirely rewritten to a Linux CI runner. The `eol=lf` normalises text in the repository; the binary entries stop git from "helpfully" corrupting images and model files.

23.5 Why `package-lock.json` is committed

`package.json` says `"three": "^0.169.0"`, which means "0.169.0 or any compatible later version". `package-lock.json` records exactly which version was installed, down to every transitive dependency.

Commit it, and `npm ci` installs precisely that set on every machine and in CI. Without it, your build depends on the day it ran.

`npm install`	Resolves versions, may update the lockfile. For adding dependencies.
`npm ci`	Installs exactly the lockfile, fails if it disagrees with `package.json`, deletes `node_modules` first. For CI and reproducible builds.

23.6 Tags and the changelog

```
git tag -a v1.0.0 -m "Caliper 1.0.0 — open a 3D model and measure it"
git push origin v1.0.0
```

A tag is a permanent pointer to one commit. Semantic versioning gives the three numbers meaning: **major** for breaking changes, **minor** for new features, **patch** for fixes. `CHANGELOG.md` is the human-readable version of the same information — grouped under *Added*, *Fixed*, *Changed*, written for someone deciding whether to

upgrade.

> **A PRACTICAL NOTE ON SQUASH MERGES**
>
> GitHub's "Squash and merge" collapses a pull request into one new commit on `main`. That commit is *not* an ancestor of your branch, so git will report the branches as diverged even though the content is identical, and `git merge --ff-only` will refuse.
>
> The habit that keeps this tidy: after each PR lands, re-point your branch at main — `git checkout dev && git fetch origin && git reset --hard origin/main`. Otherwise the divergence compounds and every future PR shows the whole history again.

CHAPTER 24

GitHub Actions, line by line

Continuous integration means a machine builds your code on every push, so "works on my machine" stops being a defence. Here is the entire workflow, explained.

24.1 What YAML is

A data format, like JSON but indentation-based. Three things to know before reading a workflow:

- **Indentation is structure.** Two spaces per level, and never tabs — a tab is a syntax error in YAML.
- `key: value` is a mapping; a leading `-` is a list item.
- `|` starts a multi-line string that keeps its newlines. This is how shell scripts are embedded in a step.

24.2 `.github/workflows/ci.yml`

```yaml
name: CI

on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main, dev]
  workflow_dispatch:
```

`on` is the trigger. This runs on pushes to either long-lived branch, on pull requests targeting them, and — thanks to `workflow_dispatch` — from a button in the Actions tab, which is invaluable when debugging.

```yaml
jobs:
  build:
    name: Typecheck and build
    runs-on: ubuntu-latest
```

A **job** runs on a fresh virtual machine. Jobs run in parallel unless one declares `needs:` another. `ubuntu-latest` is the cheapest and fastest runner.

```yaml
    steps:
      - name: Check out
        uses: actions/checkout@v4
```

The runner starts empty — it does not have your code. `checkout` clones the repository at the commit that triggered the run. Forget this step and every subsequent one fails with "file not found".

`uses` runs a published action; `@v4` pins the major version so a breaking change upstream cannot break your pipeline overnight.

```yaml
      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version-file: .nvmrc
          cache: npm
```

Reading the version from `.nvmrc` rather than hardcoding it means your machine and CI can never drift — one file, one answer. `cache: npm` caches `~/.npm` between runs, which typically cuts install time from a minute to a few seconds.

```yaml
      - name: Install
        run: npm ci

      - name: Typecheck
        run: npm run typecheck

      - name: Build
        run: npm run build
```

`run` executes a shell command. If any step exits non-zero the job fails immediately and later steps are skipped. That is the whole safety mechanism: a type error or a failed build turns the commit red.

```yaml
      - name: Upload the build
        uses: actions/upload-artifact@v4
        with:
          name: dist-${{ github.sha }}
          path: dist
          retention-days: 7
```

`${{ … }}` is expression syntax. `github.sha` is the commit hash, so each run's artifact is uniquely named and you can download exactly what CI produced rather than rebuilding and hoping it matches.

24.3 A war story worth learning from

THE PERMISSIONS WALL

An earlier version of this repository deployed to GitHub Pages. It failed on every push with `Resource not accessible by integration`.

The cause: creating a Pages site through the API requires repository *admin* rights, and the `GITHUB_TOKEN` a workflow runs with deliberately does not have them — no matter what the `permissions:` block declares. It is a security boundary, not a configuration mistake, and no amount of YAML could cross it.

Two lessons. First, when an error says "not accessible", ask what *identity* is making the request before you change settings. Second, a pipeline that fails on every push over something it cannot fix is worse than no pipeline, because people learn to ignore red. The fix was to detect the condition, explain it in the run summary, and skip — and eventually, to move to a host without that restriction.

CHAPTER 25

Deploying on Vercel

The build output is a folder of static files. Any web server can host it — Vercel just removes every step between pushing and it being live.

25.1 Why there is no deploy workflow

Vercel watches the repository through a git integration. There is no workflow file, no API token to store, and nothing in `.github/` to keep in step with the build. CI still runs and checks types — the two jobs are independent, which is exactly right: one verifies, the other ships.

25.2 Setting it up

1. Import the repository at **vercel.com/new**.
2. Vercel reads `vercel.json` and detects everything.
3. Accept the defaults.

PUSH TO	RESULT
`main`	Production deployment on the live domain
any other branch	Preview deployment on its own URL
a pull request	Preview URL commented on the PR

Preview deployments are the underrated part. Every branch gets a real, shareable URL running the real build — you can test on a phone without installing anything, and a reviewer can click rather than pulling your branch.

25.3 `vercel.json`

```
{
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",

  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],

  "headers": [ … ]
}
```

The rewrite

Serve `index.html` for any path that is not a real file. As covered in 17.3, this is what stops deep links 404ing in a single-page application. Vercel checks the filesystem *before* applying rewrites, so `/assets/index-abc.js` still serves the real file.

Cache headers

```
{
  "source": "/assets/(.*)",
  "headers": [{ "key": "Cache-Control",
                "value": "public, max-age=31536000, immutable" }]
},
{
  "source": "/index.html",
  "headers": [{ "key": "Cache-Control",
                "value": "public, max-age=0, must-revalidate" }]
}
```

One year for hashed assets, and `immutable` tells the browser not even to send a revalidation request. Zero for the HTML shell. This pairing is the whole caching strategy of a modern SPA, and it only works because the asset filenames contain content hashes.

25.4 Environment variables

Caliper needs none — it has no backend. When you do need them, Vite exposes only variables prefixed `VITE_` to client code:

```
const key = import.meta.env.VITE_API_URL;
```

> **ANYTHING IN A FRONTEND BUNDLE IS PUBLIC**
>
> The prefix requirement exists to stop you leaking server secrets into client code by accident. But understand the real rule: **a value compiled into a JavaScript bundle can be read by anyone who opens dev tools.** Public API endpoints and publishable keys are fine. Private keys are never fine, in any framework, under any variable name.

CHAPTER 26

Security headers and the CSP

Five headers, set once in `vercel.json`, that close off entire categories of attack.

26.1 The simple four

HEADER	WHAT IT PREVENTS
`X-Content-Type-Options: nosniff`	Stops the browser guessing a file's type from its contents. Without it, an uploaded "image" that is really JavaScript can be executed.
`X-Frame-Options: DENY`	Stops your site being embedded in an `iframe` on someone else's page — the basis of clickjacking.
`Referrer-Policy: strict-origin-when-cross-origin`	Sends only your domain, not the full URL, when a visitor follows a link away. Full URLs can leak identifiers.
`Permissions-Policy: geolocation=(), camera=(), …`	Switches off browser features the app never uses. If a dependency is ever compromised, it still cannot ask for the camera.

26.2 Content Security Policy

The important one, and the one worth understanding properly. A CSP is an allow-list of where content may be loaded from. Anything not listed is blocked by the browser, which makes it the strongest defence there is against cross-site scripting.

```
default-src 'self';
script-src  'self' 'unsafe-inline' 'unsafe-eval' 'wasm-unsafe-eval'
            https://cdn.jsdelivr.net https://www.gstatic.com;
style-src   'self' 'unsafe-inline' https://fonts.googleapis.com;
font-src    'self' https://fonts.gstatic.com;
img-src     'self' data: blob:;
connect-src 'self' data: blob: https:;
worker-src  'self' blob:;
object-src  'none';
base-uri    'self';
form-action 'none';
frame-ancestors 'none'
```

Read it as a series of questions about this specific application:

`default-src 'self'`	The fallback: same origin only, unless a more specific rule says otherwise.
`script-src … jsdelivr, gstatic`	The CAD and BIM kernels are loaded from jsDelivr; the Draco decoder from Google's static host. **Without these two entries the CAD formats silently stop working.**
`'wasm-unsafe-eval'`	Compiling WebAssembly counts as evaluation, and this permits it specifically.
`'unsafe-eval'`	Required, and it was learned the hard way — see below.
`'unsafe-inline'`	An honest compromise. The theme script in the head is inline and must run before paint. The stricter alternative is a per-build nonce or hash, which is worth doing on an application handling sensitive data.
`img-src … data: blob:`	Screenshots become `data:` URLs; dropped files become `blob:` URLs. Both are needed for the app to function.
`connect-src … https:`	Deliberately open, because `?model=` may point at any host the visitor chooses.
`object-src 'none'`	No Flash, no applets. Free win.
`frame-ancestors 'none'`	The modern replacement for `X-Frame-Options`. Both are set for older browsers.

A CSP BUG THAT ONLY EXISTED IN PRODUCTION

This policy originally carried `'wasm-unsafe-eval'` without `'unsafe-eval'`, on the reasonable-sounding theory that the app only needs to compile WebAssembly and never to evaluate a string.

Every CAD and BIM format broke. Opening a `.step` file gave "*Evaluating a string as JavaScript violates the following Content Security Policy directive*". The Emscripten runtimes these kernels are built with do evaluate strings — `rhino3dm` calls `new Function` twice — and Chrome reports blocked WebAssembly through the same message, so the two causes are indistinguishable from the console alone.

What makes it worth studying is **where it could be caught**. Unit tests cannot see it: the code is correct. `npm run dev` cannot see it: the dev server sends no headers. `npm run preview` cannot see it either — same files, still no headers. It appeared only on the deployed site, in the one configuration nothing tested.

The fix was two-part: add `'unsafe-eval'`, and add `e2e/serve-production.mjs`, which serves `dist/` with the headers read straight out of `vercel.json` so the sweep in `e2e/formats.mjs` exercises the real policy. It cannot drift from production, because it has no configuration of its own.

HOW TO WRITE A CSP WITHOUT BREAKING YOUR APP

Start with `Content-Security-Policy-Report-Only`, which logs violations to the console without blocking anything. Use the site normally, exercise every feature, collect the violations, then widen the policy to cover exactly those and switch to enforcing mode. Writing one from scratch and enforcing it immediately will break something you did not think of — in this app it was the CAD kernels, which most people never load.

PART V

Learning from it

Ten exercises that take you from a one-line change to a whole new feature, the mistakes this codebase was written to avoid, and where to read next.

CHAPTER 27

Ten exercises

In rough order of difficulty. Do them on a branch. Break things deliberately — a concept you have broken and fixed is one you own.

1 · Change some copy — 5 minutes

In `src/components/Panel.tsx`, change `"Depth"` to `"Length"`. Save and watch it update without a reload.

Learns: hot module replacement, where interface text lives.

2 · Add a unit — 15 minutes

Add feet. You will need to touch `UnitSystem` in `src/types.ts`, `UNIT_FACTOR` in `src/lib/format.ts`, and `UNITS` in `Panel.tsx`. One foot is 304.8 mm.

> **DO THIS FIRST**
>
> Add `'ft'` to the type and try to build *before* changing anything else. TypeScript will list every place that must be updated. That experience — the compiler handing you a to-do list — is the single best argument for typed code, and you should feel it once early.

Learns: union types, exhaustiveness, how a change ripples.

3 · Add a keyboard shortcut — 15 minutes

Make `T` toggle the details panel. Open `src/hooks/useHotkeys.ts`, find the `switch`, and add a case calling `store.togglePanel()`. Then check the guards: does it still work when a button has focus? Does it fire while you are typing in the unit dropdown?

Learns: global event handling, reading state without subscribing.

4 · Change the accent colour — 20 minutes

Make the app teal. Edit only `--accent` and its variants in `src/styles/tokens.css` — both themes. Then find the one place the 3D scene needs telling too: `SKIN.mark` in `src/viewer/Stage.ts`, because three.js materials cannot read CSS variables.

Check contrast afterwards. Does `--accent-ink` still read on your new colour in both themes?

Learns: design tokens, the CSS/WebGL boundary, contrast.

5 · Show the surface area — 45 minutes

Add total surface area to the size section. In `src/viewer/analyze.ts`, walk every triangle and sum the area of each — half the length of the cross product of two of its edges. Add the field to `SceneStats`, then render it in `Panel.tsx`.

Learns: reading geometry buffers, vector maths, extending a type end to end.

6 · Bring back the angle tool — 2 hours

The most instructive exercise here, because it touches every layer. You need: a new value in `MeasureTool`; three points instead of two in `POINTS_NEEDED`; an `angleAt` function using the dot product; an arc drawn in `Annotations.ts`; a card in the panel; and a keyboard shortcut.

```ts
export function angleAt(from, vertex, to): number {
  const u = from.clone().sub(vertex);
  const v = to.clone().sub(vertex);
  const lengths = u.length() * v.length();
  if (lengths < 1e-12) return 0;
  const cosine = THREE.MathUtils.clamp(u.dot(v) / lengths, -1, 1);
  return THREE.MathUtils.radToDeg(Math.acos(cosine));
}
```

Note the `clamp`. Floating-point error can push the cosine a hair past 1, and `Math.acos(1.0000001)` is `NaN` — which then propagates silently into the interface. Guarding degenerate cases is the recurring theme of every geometry function in this project.

Learns: the full vertical slice — type, engine, drawing, UI, keyboard.

7 · Add a format — 2 hours

Add `.xyz` point clouds: a plain text file of `x y z` per line. Write `src/viewer/loaders/xyz.ts` following the shape of `off.ts`, add a row to `FORMATS`, and a `case` in the dispatcher. Use `THREE.Points` with `PointsMaterial` rather than a mesh.

Learns: designing to an interface, why the loader contract exists.

8 · Persist measurements — 2 hours

Save measurements to `localStorage` keyed by filename, and offer to restore them when the same file is opened again.

The interesting part is not the storage — it is the design question. Is the filename enough to identify a model? What if the file changed? Should you hash the contents instead? There is no single right answer, and thinking it through is the exercise.

Learns: persistence, identity, designing under ambiguity.

9 · Add a second route — 3 hours

Install `react-router-dom` and add an `/about` page, keeping the viewer at `/`. Follow chapter 17. Then build for production, serve `dist/` with a plain static server, and load `/about` directly — watch it 404, and understand why the rewrite in `vercel.json` exists.

Learns: client routing, and the deployment trap that comes with it.

10 · Build your own — a weekend

Start a new Vite + React + TypeScript project and rebuild one slice of this from scratch: a canvas, a cube, orbit controls, and a panel showing its dimensions. No copying — refer back only when stuck.

You will get it wrong in interesting ways. The engine will leak because you forgot cleanup. The panel will re-render constantly because you selected the whole store. The canvas will be blurry because you forgot `setPixelRatio`. Every one of those is a paragraph in this document that will suddenly mean something.

Learns: everything. This is the exercise that matters.

CHAPTER 28

Mistakes this codebase avoids

Every one of these was either found and fixed here, or deliberately designed around. They are the most portable thing in this document.

28.1 In React

MISTAKE	INSTEAD
Subscribing to the whole store	Select the narrowest slice. A component that shows the unit should not re-render when the frame rate changes.
Effects with no cleanup	Anything created must be destroyed — listeners, timers, animation frames, WebGL contexts.
Array index as a list key	A stable id from the data.
Mutating state	Create a new array or object. React compares by identity.
Hooks after an early return	All hooks first, then the conditional return.
`{count && <X />}`	`{count > 0 && <X />}` — otherwise a literal `0` renders.
State for DOM references	`useRef`.

28.2 In three.js

MISTAKE	INSTEAD
Never disposing	GPU memory is not garbage collected. Dispose geometries, materials, textures.
Rendering every frame regardless	Render on demand; invalidate when something changes.
`scene.overrideMaterial` for render modes	It repaints the whole scene including your helpers. Swap materials on the model only.
Global clipping planes	They cut the grid and the floor too. Set them per material.
Ignoring `webglcontextlost`	Handle it and call `preventDefault()`, or the canvas is black forever.
Allocating vectors in hot loops	Reuse module-level scratch objects.
Selecting after a drag	Compare pointer-down and pointer-up positions.

28.3 In CSS

MISTAKE	INSTEAD
`100vh` on mobile	`100dvh` — `vh` includes the space behind the address bar.
`backdrop-filter` over a live canvas	Measured at 331 ms per frame here. Use it on small static chips only.
Removing focus outlines	`:focus-visible` gives you a clean mouse experience and a usable keyboard one.
Hardcoding both halves of a colour pair	Token the background *and* the ink that sits on it.
Ignoring `prefers-reduced-motion`	Four lines. No excuse.
A separate `.is-active` class	Style from `aria-pressed` so state and appearance cannot drift.

28.4 In architecture

MISTAKE	INSTEAD
Writing the same fact twice	Derive it. The "18 formats" bug — a hand-written list beside a registry of 22 — is exactly this.
Two sources of truth for one thing	Visibility lived in both the store and the scene graph once; the tree's icons went stale. One owner.
Adding a library before the problem	Let the problem ask for the tool. No router until there are routes.
Silent failure	If a circle cannot be fitted, say why and say what to do instead.
Fake progress bars	`null` means indeterminate. Never invent a percentage.
Measuring after transforming	Take the file's real dimensions before normalising it for display.

28.5 The habit underneath all of them

Nearly every entry above comes from the same discipline: **know which thing owns which fact.** One owner for the colour, one for the format list, one for visibility, one for the measurement units. When two pieces of code both believe they are responsible for something, they will disagree — not today, but on the day someone edits one of them.

CHAPTER 29

Glossary

Frontend and React

Component	A function returning markup.
Props	A component's arguments. Flow downward only.
State	A value that causes a re-render when changed.
Hook	A function starting with `use` that taps into React's lifecycle.
Ref	A mutable box that survives renders without causing them.
Effect	Code that runs after render, usually to touch something outside React.
JSX	HTML-like syntax that compiles to function calls.
Prop drilling	Passing props through components that do not use them.
Selector	A function picking one slice of a store, so only that slice triggers re-renders.
Controlled input	A form element whose value comes from state.
Error boundary	A class component that catches render errors below it.
FOUC	Flash of unstyled content — the wrong theme appearing before the right one.
SPA	Single-page application: one HTML file, JavaScript swaps the content.
HMR	Hot module replacement — swapping a changed module without reloading.
Tree shaking	Dropping code nothing imports.
Code splitting	Producing several bundles so the browser loads only what it needs.

3D graphics

WebGL	The browser's low-level GPU drawing API.
Mesh	Geometry plus material — one visible object.
Geometry	Arrays of vertex positions, normals and indices.
Material	How a surface responds to light.
Vertex	A point in space; a triangle has three.
Normal	A vector perpendicular to a surface, used for lighting.
Index buffer	A list of vertex numbers, so vertices can be shared between triangles.
Raycasting	Firing a ray to find what it hits — how clicking works in 3D.
NDC	Normalised device coordinates: the −1…+1 square the camera projects into.
Bounding box	The smallest axis-aligned box containing an object.
Scene graph	The tree of objects; transforms are inherited by children.
Tessellation	Converting mathematical surfaces into triangles.
B-rep	Boundary representation — how CAD stores exact shapes.
PBR	Physically based rendering — the metalness/roughness model.
Tone mapping	Compressing a wide brightness range into what a screen can show.
Z-fighting	Flickering where two surfaces are too close for depth precision.
Draw call	One instruction to the GPU. Fewer is faster.

Tooling and deployment

Bundler	Combines many source files into few optimised ones.
Transpile	Convert one language version to another — TypeScript to JavaScript.
Content hash	A filename fingerprint derived from contents, enabling permanent caching.
Source map	A file letting dev tools show original source for minified code.
CI	Continuous integration — automated checks on every push.
Artifact	A file a CI run produces and stores.
Rewrite	Serving one path's content for another, without changing the URL.
CSP	Content Security Policy — an allow-list of where content may load from.
CORS	The rules governing cross-origin requests.
Semantic versioning	`MAJOR.MINOR.PATCH` — breaking, feature, fix.
Lockfile	A record of exactly which dependency versions were installed.
WebAssembly	A compiled binary format that runs at near-native speed in the browser.
Blob URL	A temporary `blob:` URL pointing at in-memory data.
PWA	A web app installable like a native one, via a manifest.

CHAPTER 30

Where to read more

30.1 Documentation worth actually reading

MDN Web Docs `developer.mozilla.org`	The reference for HTML, CSS and JavaScript. Not a tutorial site — the place you go to find out precisely what something does. Bookmark it.
React `react.dev`	The rewritten docs are genuinely excellent. Read "Escape Hatches" in particular — it covers refs and effects, which is where most React confusion lives.
three.js `threejs.org`	The examples page is the real documentation. Every example has viewable source, and reading three of them teaches more than any tutorial.
Vite `vite.dev`	Short and clear. The "Features" page explains what you get for free.
TypeScript Handbook `typescriptlang.org/docs/handbook`	Read "Everyday Types" and "Narrowing" and you will have most of what daily work needs.
Zustand `github.com/pmndrs/zustand`	The README is the documentation, and it is short enough to read in one sitting.
web.dev `web.dev`	Google's performance and best-practice guides. "Learn CSS" and "Learn Accessibility" are both solid courses.

30.2 A suggested order

1. **Solidify JavaScript first.** Frameworks are much easier once closures, promises, `this`, destructuring and array methods are second nature. Most "React is confusing" is really "JavaScript is confusing".
2. **Then React fundamentals** — components, props, state, effects. Build three small things without a router or a state library.
3. **Then TypeScript**, applied to something you already wrote in JavaScript. Converting a working project teaches more than starting typed.
4. **Then the platform** — CSS Grid, custom properties, accessibility, the Fetch API, the browser's own capabilities. This is what separates developers who can only work inside a framework from those who can work anywhere.
5. **Then tooling** — Vite, git, CI, deployment. It is less glamorous and it is what makes you employable.
6. **Then a specialism**, if you want one. 3D, data visualisation, animation, editors. Depth in one area beats shallow familiarity with ten.

30.3 Three habits worth more than any framework

Read source code. Not tutorials — real repositories. You have just read one end to end; that is a skill, and it compounds. When a library surprises you, open its source. It is almost always less magic than you expect.

Measure before you optimise. The 331 ms frame in chapter 18 was invisible until someone timed it, and no amount of reading would have found it. Open the performance panel. Time things. Guessing about performance is how weeks get wasted on the wrong problem.

Write down why. Not what — the code says what. Why this approach, what you tried first, what will break if it is changed. Every non-obvious decision in this codebase has a comment explaining itself, and that is the difference between a project someone can pick up and one they have to reverse-engineer.

> **LAST THING**
>
> The application this document describes is about four thousand lines. It is not big. What makes it worth studying is not its size but that every line had a reason, several of the reasons were wrong and got fixed, and the fixes are documented. Build something small and finish it properly. That teaches more than starting something ambitious and abandoning it — and a finished small thing is also the only kind you can show anyone.

APPENDIX

Quick reference

A · Keyboard

KEY	ACTION
`Q` `W` `E`	Shaded · wireframe · shaded with edges
`1`–`6`, `0`	Front, back, left, right, top, bottom, isometric
`F`	Fit the model to the screen
`G` `S`	Build plate · shadows
`M` `D`	Measure distance · measure diameter
`Backspace`	Undo the last measuring point
`Esc`	Put the tool away, then clear the selection
`⌘/Ctrl O` `⌘/Ctrl S`	Open a model · save a PNG
`⌘/Ctrl B` `⌘/Ctrl J`	Details panel · theme

B · Commands

`npm install`	Install dependencies
`npm ci`	Install exactly the lockfile (CI)
`npm run dev`	Dev server with hot reload
`npm run typecheck`	Type-check only
`npm test`	Unit tests (Node's built-in runner)
`npm run build`	Type-check, then build to `dist/`
`npm run preview`	Serve the production build locally

C · Where to change what

TO CHANGE…	EDIT
Any colour or font	`src/styles/tokens.css`
Colours inside the 3D scene	`SKIN` in `src/viewer/Stage.ts`
Panel content or layout	`src/components/Panel.tsx`
Tool rail buttons	`src/components/ToolRail.tsx`
Keyboard shortcuts	`src/hooks/useHotkeys.ts`
Supported formats	`FORMATS` in `src/lib/formats.ts`
Camera, picking, render modes	`src/viewer/Engine.ts`
Measuring logic	`src/viewer/measure.ts`
Application state and actions	`src/store/useViewer.ts`
Page title, meta tags, theme script	`index.html`
Build settings	`vite.config.ts`
Deployment and headers	`vercel.json`
CI pipeline	`.github/workflows/ci.yml`